Report and Documentation
For Project II of CS505

By AKM Sabbir

Date: 04/27/2014

This project is about to simulate the distributive operations those can be performed over a database that is distributive in its architecture. Designing such a database is not a trivial task and designing the optimal queries which will perform different manipulative and definitive operations on this database is not a trivial task also. However in order to understand the underlying principal of such a kind of database that is how they are built and how different operations performed over this; we are considering a very small problem that can be solved with in our limited time and by using our initial level knowledge about it. So the problem definition is like below,

The database has three schemas they are branch, customer and employee. They have total 12 attributes denoting different type of information. They are distributed among four different local sites. So four local branches have the same tables but contain different local information. Now when n user login to the system in global mode s/he should be able to make query for all local branches and retrieve the necessary information from them. And we would like to implement the map reduce technique to implement the parallel nature of simultaneous information retrieval from different individual copies.

So I have used Python 2.7 for designing and implementing the front end UI, control logic and mySql 5.6 for back end database design. To build the UI I have used wxPython module of python but there are other UI technologies are also available in python for doing GUI programming such as,

- Tkinter
- Qt
- Gtk

I opted to use wxPython as it is very widely used and easy to learn.

I will maintain the MVC (Model/View/Control) pattern to describe my solution about this problem. So the top level outline is,

1. Data Model: this will describe the underlying database schemas over which the business logics have been implemented.
2. Controller: the business logic that have been implemented using the python technology modules like Threading, re, MySQLdb.
3. View: wxPython module has been used to implement the user interface details.

I am going to describe all of these parts in details in later part of the report. This desktop based application solution is executable in Linux environment also.

So first I am going to give description of the database for the application. Actually the database used quite a bit of some advanced concept of SQL language.

Data Model:

I have the following databases in my local mysql 5.6 database server.

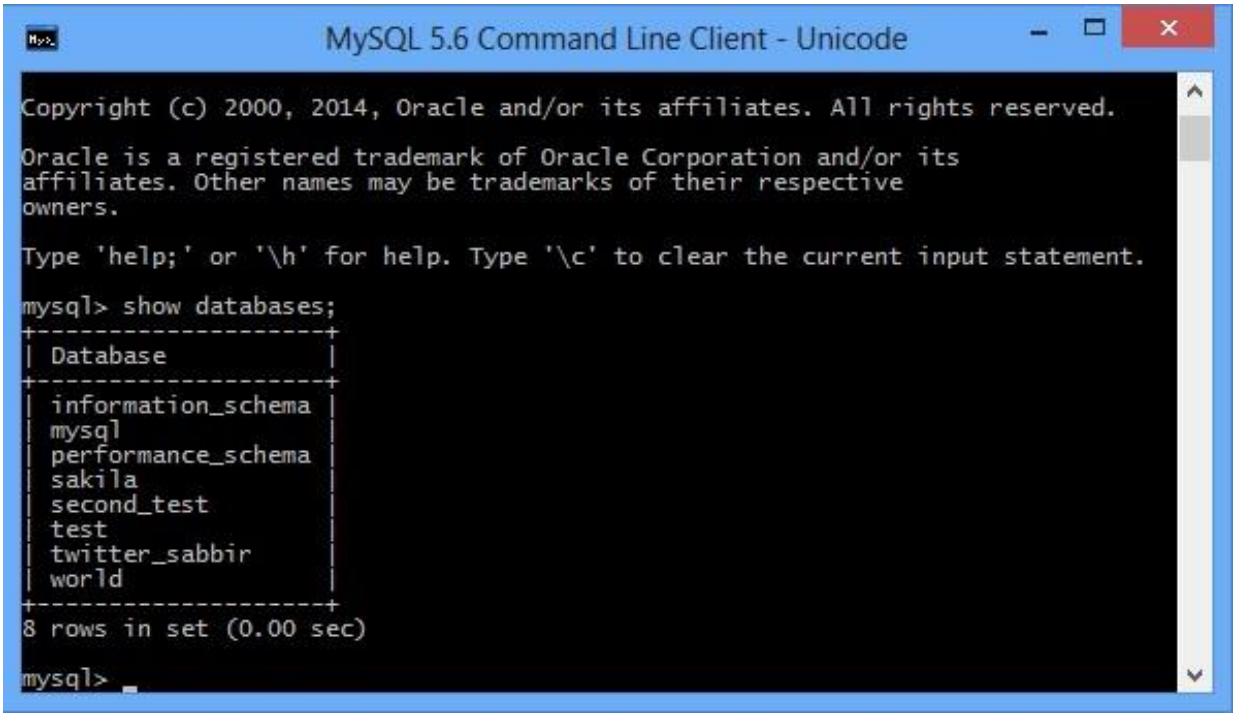


Fig: 1

It has total 8 databases created for different purpose. The database of interest for our application is twitter_sabbir. I have created all the necessary schemas in this database for our application. The schemas in this database are followings,

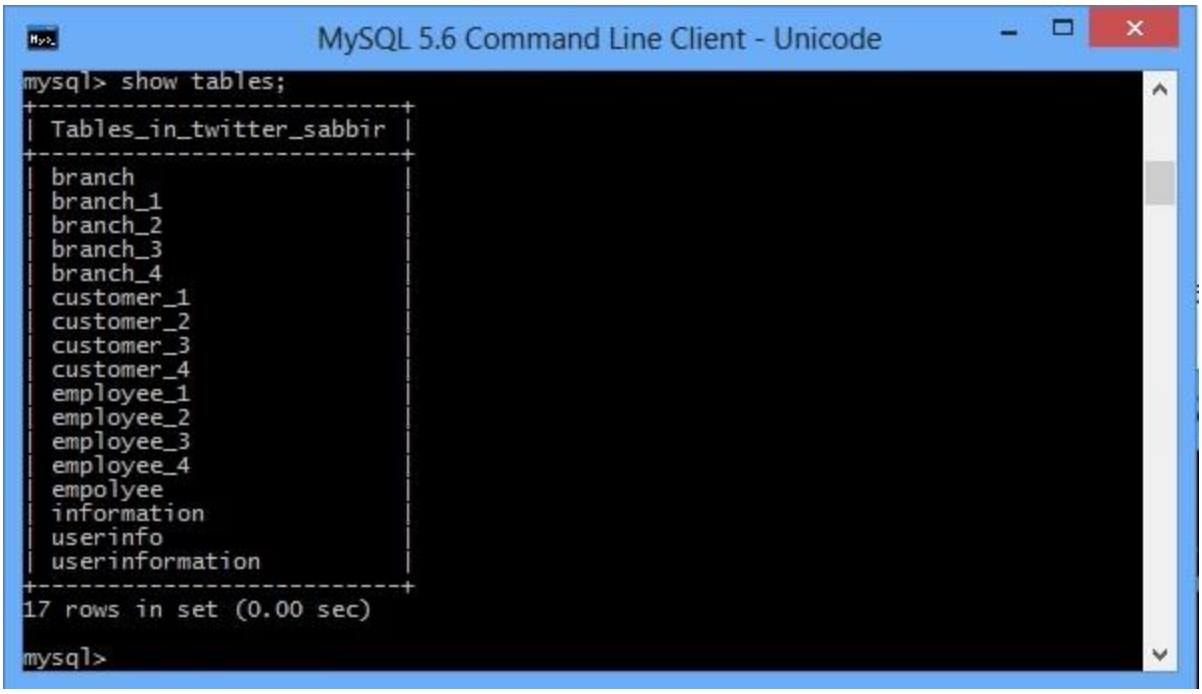


Fig: 2

It has 17 different schemas. But for my application I used just 10 tables. Four employee tables for four different sites similarly four customer tables for four different sites; one branch table containing global branch information, and one userInformation table containing user information for authentication purpose. So the table information is as follows:

- Employee_{1-4}: this able has five attributes, employeeID, branchID, empfname,emplname, salary. Where employeeID is the primary key and branchID is the foreign key constraint. I used four of them to represent four different sites.
- Customer_{1-4} this table has four attributes customerID, branchID, acctype and balance. customerID is primary key and branchID is foreign key constraint. I used four of these tables to simulate four different sites.
- Branch: branch table contains three attributes. Among them branchID is the primary key. I used only one copy of it. Its unnecessary to maintain four copies of it because simple queries would retrieve four same copies from four different sites. That mean maintaining four different branch tables are redundancy.
- userInformation: this table contains username, password, branchID attributes. I combined this information to form a user object so that I can perform necessary local or global query operations over database for that specific user.

In the child Tables I have used on delete cascade and on update cascade constraint statement while creating tables so that the child tables remain consistent with the parent table even after deletion and update operation.

User Interface Model:

In the user interface model, I used two separate frames. One for log in that is the inception point of the application and is of the following appearance,

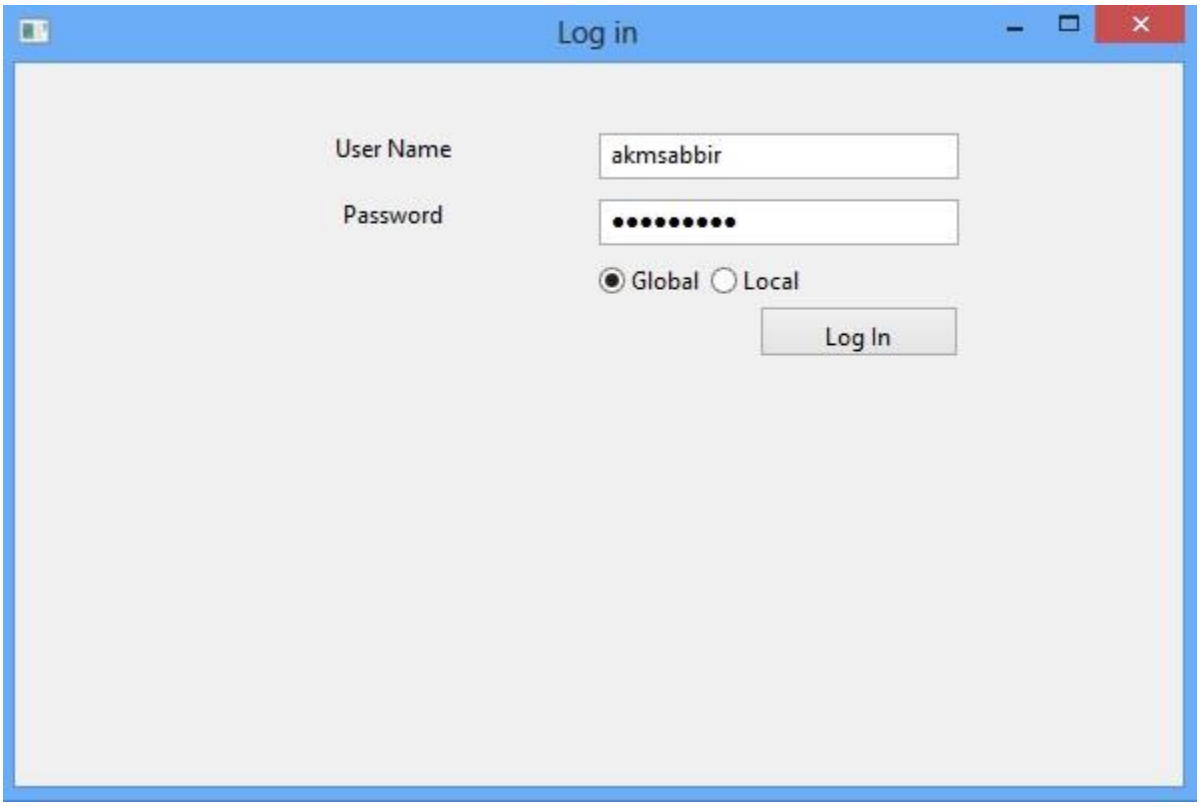


Fig: 3

It has two text fields to mention the username and password while user try to login. User can also mention the mode in which s/he wants to login. Depending on the mode the appearance of subsequent form will be different. This form will establish connection with database and will use the data from userInformation table to verify the log in information

Then I have the following main form contains all the necessary fields to form a valid query for my business logic,

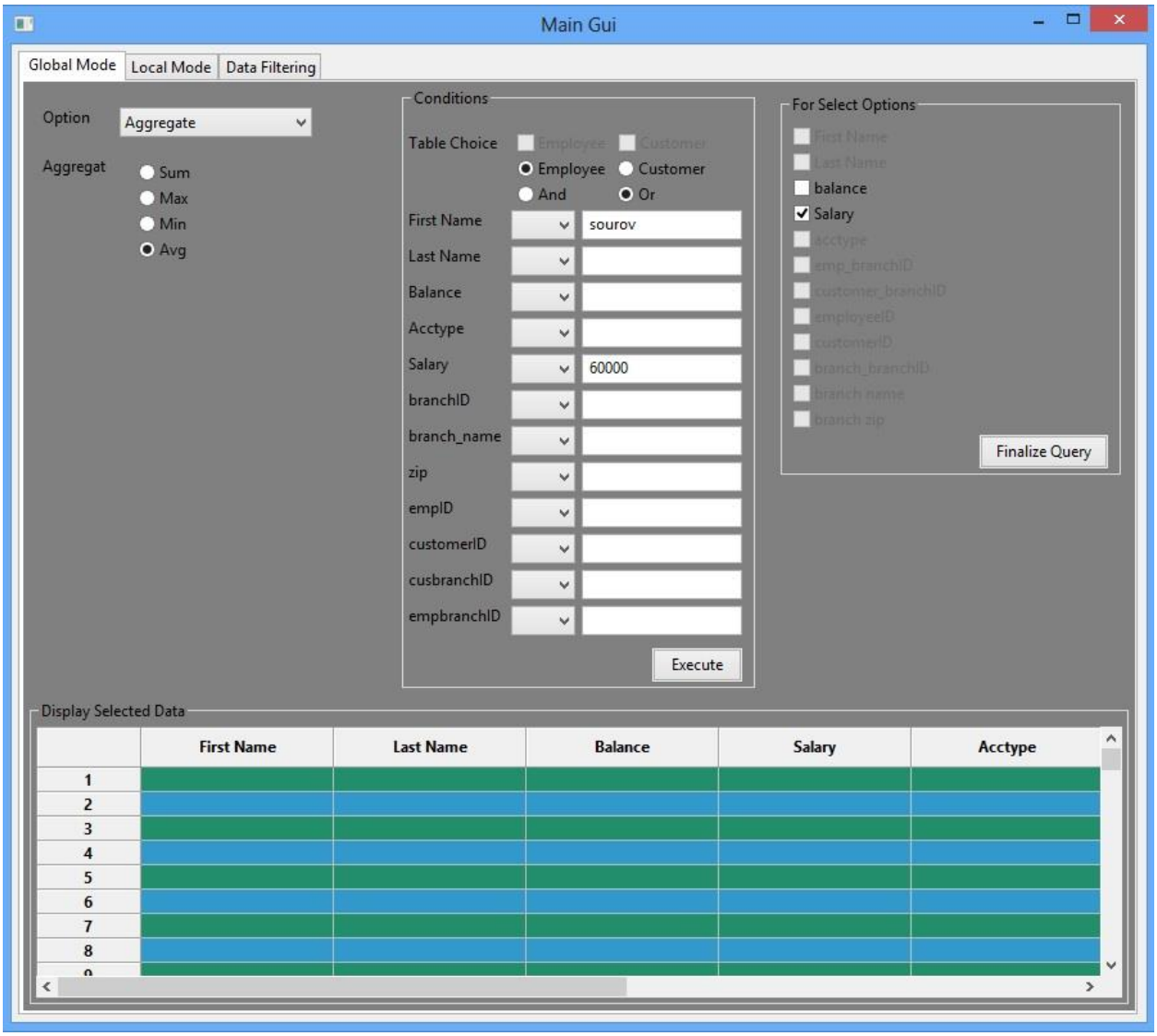


Fig: 4

As you can see this main form contains three tab panes. One is for global mode; second one is for local mode and another one is for data filtering. Although a user can switch between local and global pane, a user in local mode cannot form a query in global mode as restriction logic has been implemented.

Now I am going to describe different components of this global Mode pane.

Option and Aggregate: option is a combo box can take the following values

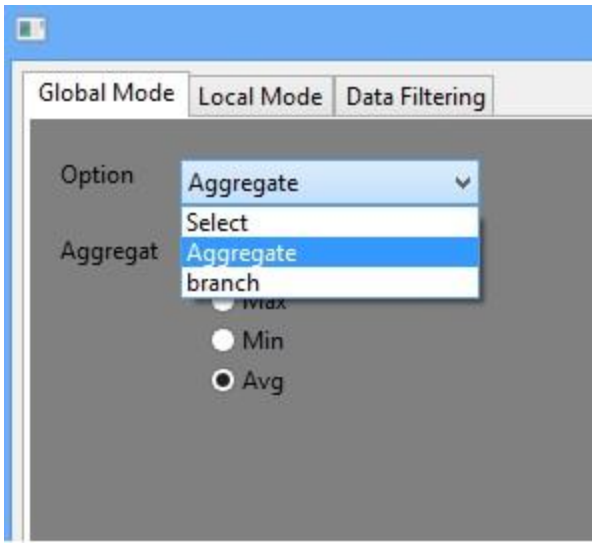


Fig: 5

'Select' signifies that you are going to form and use the simple Select queries.
Aggregate signifies that you are going to form and use the simple Aggregate queries.
Branch here is used for different purpose. In global mode one can only apply the simple queries. However branch is table contains nominal attributes except branchID, so this option value allows user to apply simple queries to branch table by choosing the branch option value from here.

Aggregate option gives the user the option of choosing an aggregate function to apply it on the query. There are only four aggregate functions can be chosen from sum, max, min, avg.

Conditions: this condition static box area has the options of choosing table values. The table values are customer and employee. These two tables are distributive table, and they have four copies at four different sites. One can apply simple queries by selecting them.

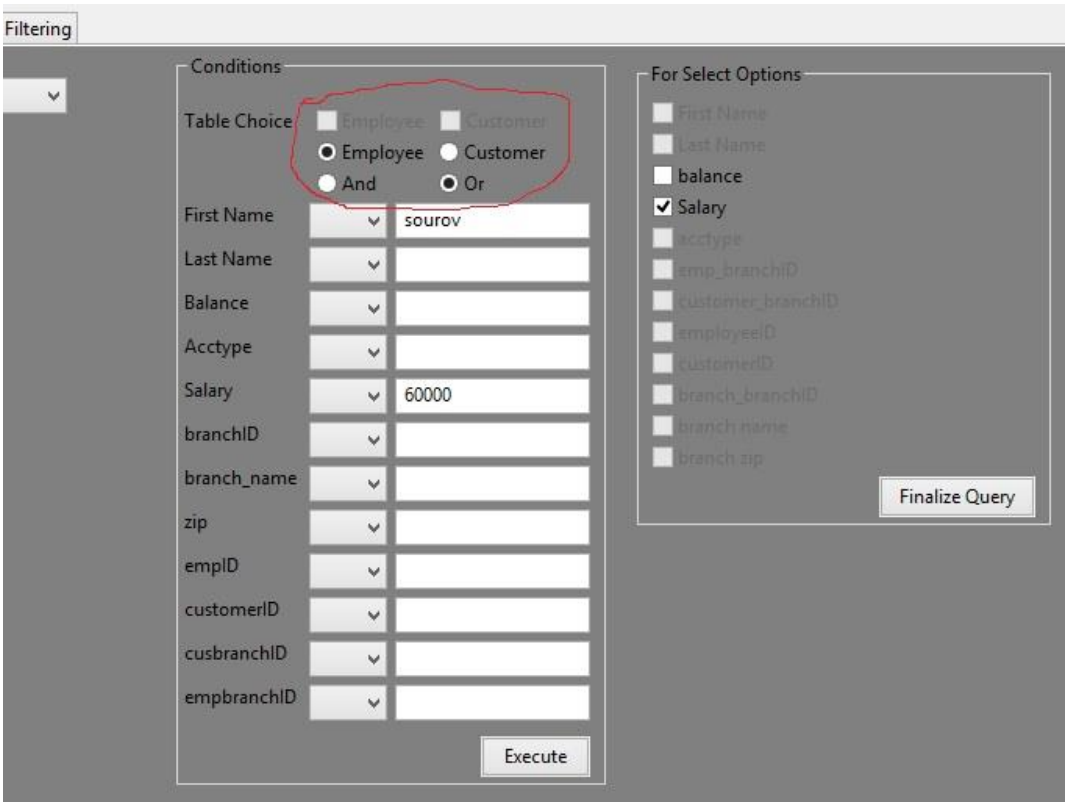


Fig: 6

So the table choice part has two check boxes. These check boxes will be activated when a user will choose select option value. So user can join tables at select mode. And the Radio Button option will be activated when user will choose the Aggregate option value. So aggregate function can only be applied in one table at a time. It also does not very useful to join this two tables for aggregate operation. One can serially execute separate queries on these two tables to get the required aggregate results.

Then it contains two more radio buttons And and or. This is for creating the combination of conditions. One can put several conditions with and, or conjunctions.

It also contains twelve conditional combo box and conditional text fields. The conditional combo box has the following values,

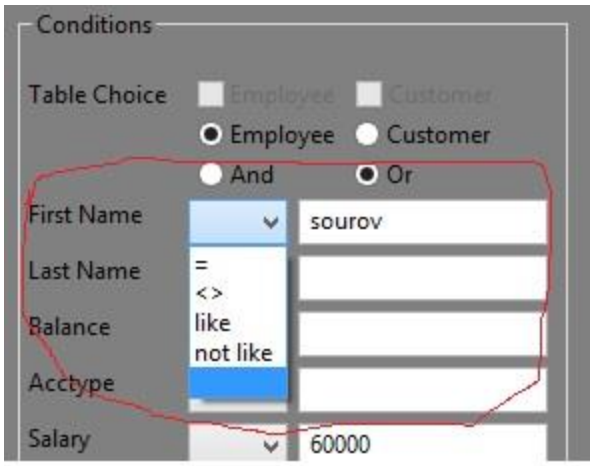


Fig: 7

So, user has to select one of these values to activate the conditional field. Then put the conditional value at the text field. User need to fill up both the field otherwise the conditional field will not be activated.

Finally it has the execute button. This button execute the finalize query. A query has to be finalized before it can be executed.

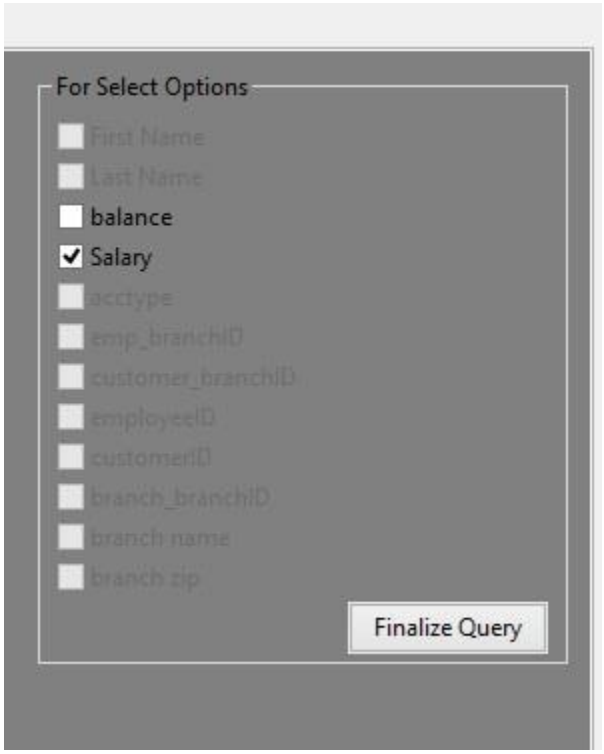


Fig: 8

Select: Finally important select field. This section contains all the 12 attributes mentioned in all the schemas. Based on appropriate table selection the corresponding attributes will be activated so that user can select them to form the select portion of the query. Then the finalize button, every time user wants to change the query, s/he needs to first apply the changes then press the finalize query button to verify the formed query. For example after pressing the finalize query following result will show up.

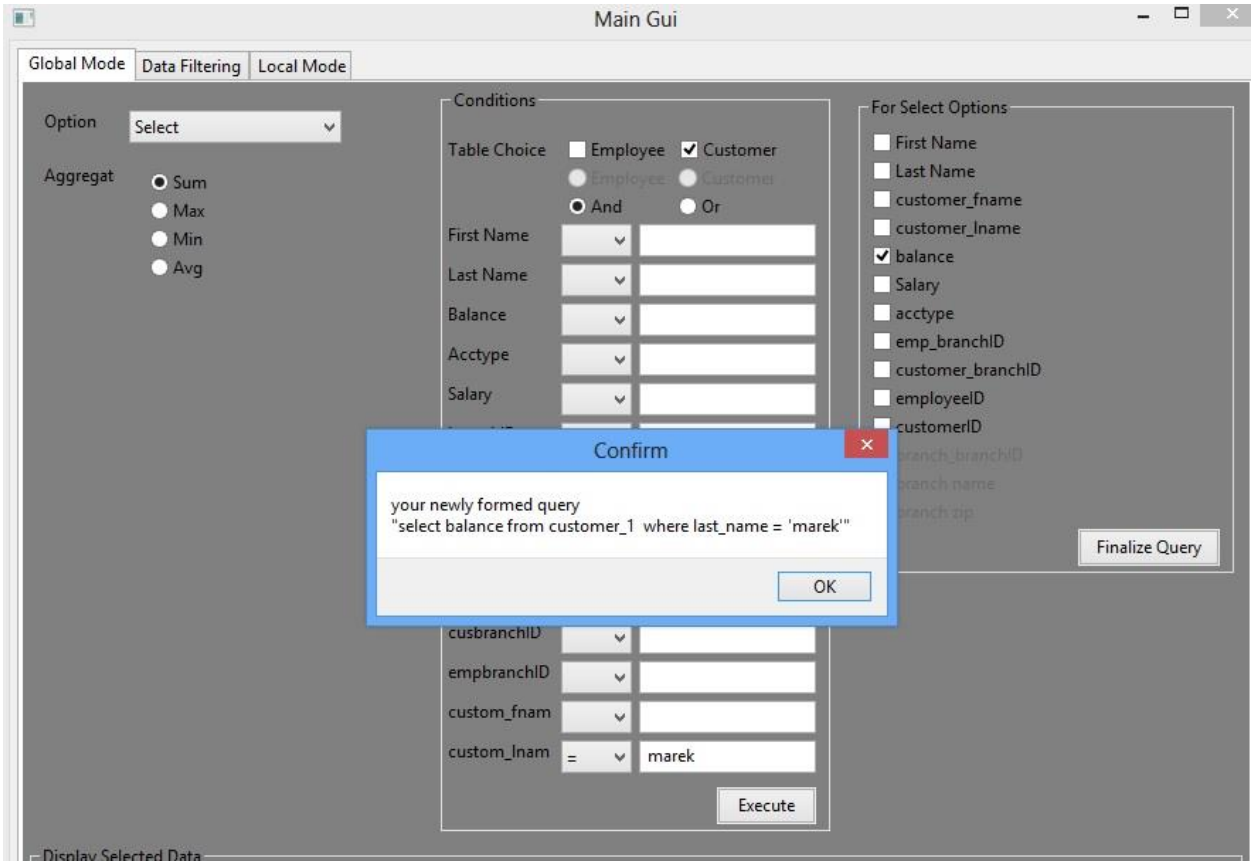


Fig: 9

The message box shows the newly formed query. So user can verify whether the newly formed query is syntactically correct or wrong. If the query is correct then one can press the execute button. After that the result of executed query will be shown in the either grid view or in dialog box such as, if it's a select query it will be displayed in grid and if its aggregate query then result will be shown in dialog box. Below images is going to clarify this,

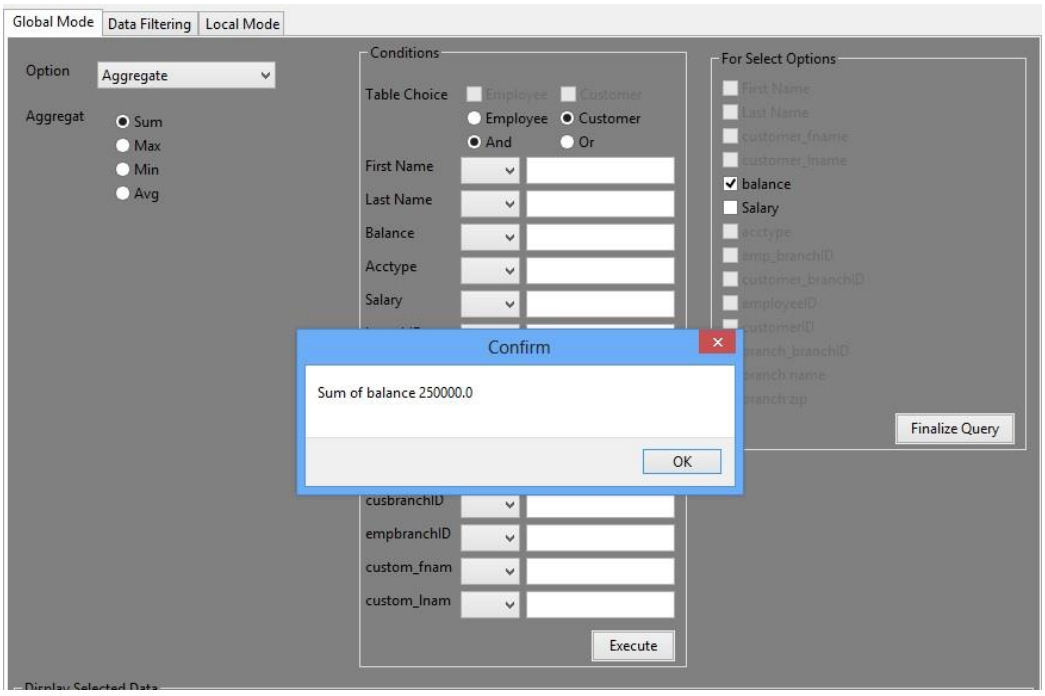


Fig: 10

Aggregate result divulged in a dialog box.

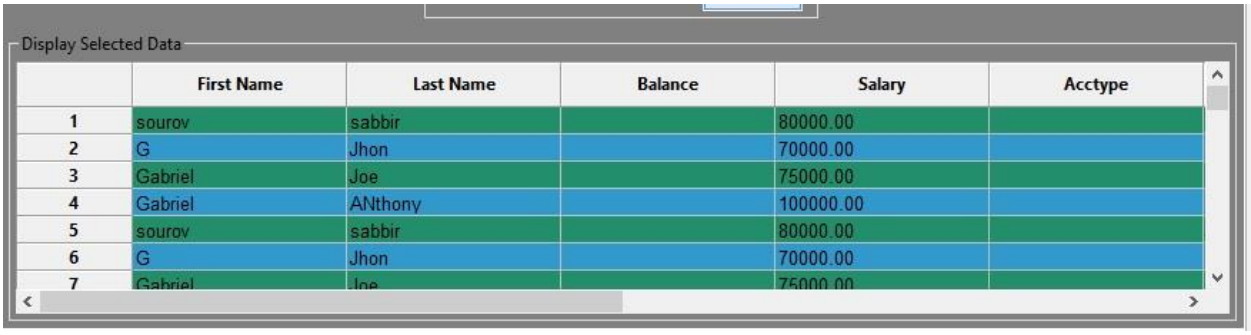
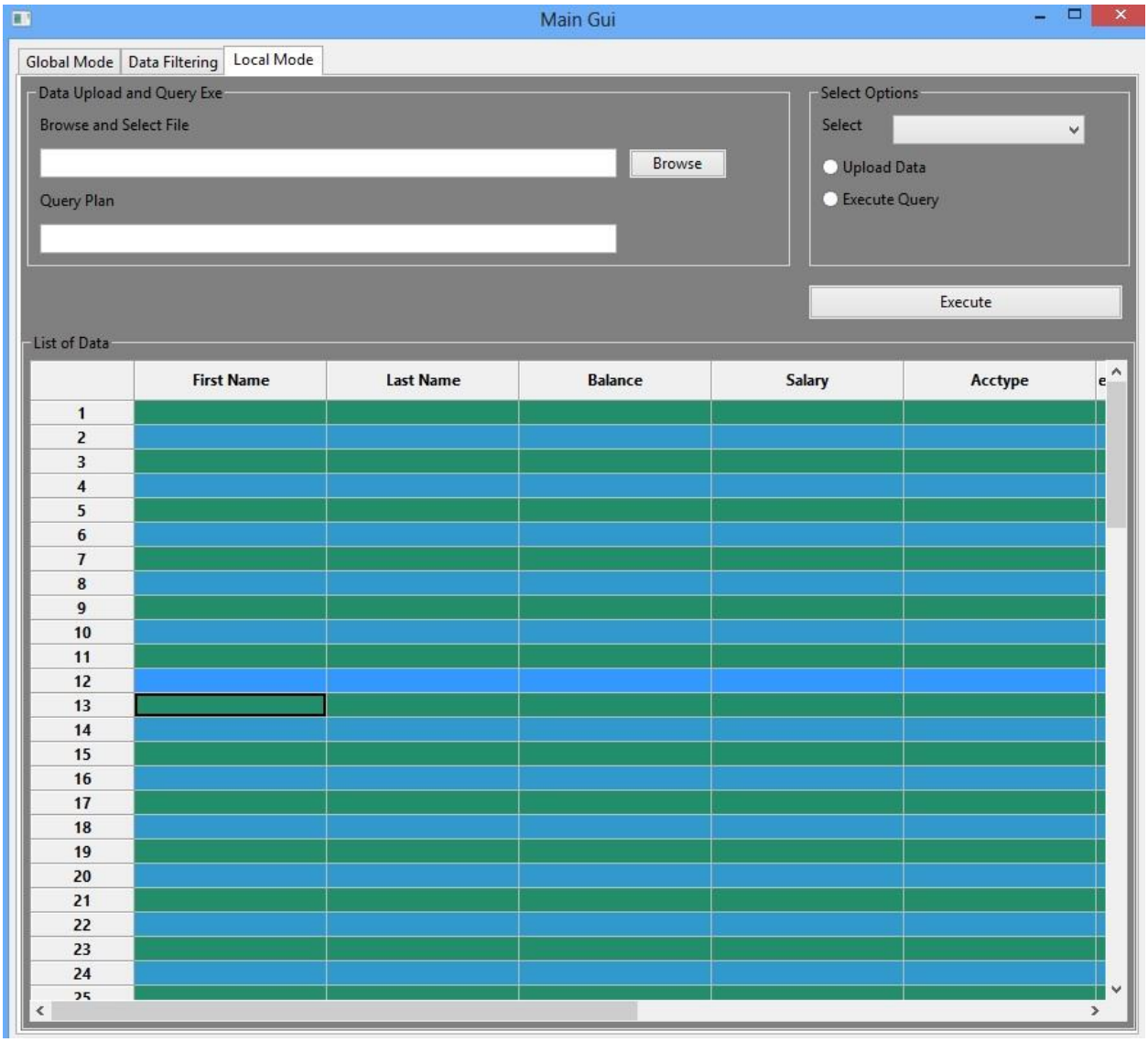


Fig: 11

Result of select operation populated in a grid view.

Local Mode:

In local mode the user will get a view like the following image



This view has four important components they are,

- Data upload and Query Exe
- Select Option
- Execute
- List of Data Grid View

Data upload and Query Exe:

Browse and select file component allows user to browse the system directory and select an appropriate file to load the data into the database table at once. User can upload large data very quickly by using this option.

And the query plan text field allows user any query to execute at the local mode it includes select, update, insert, delete.

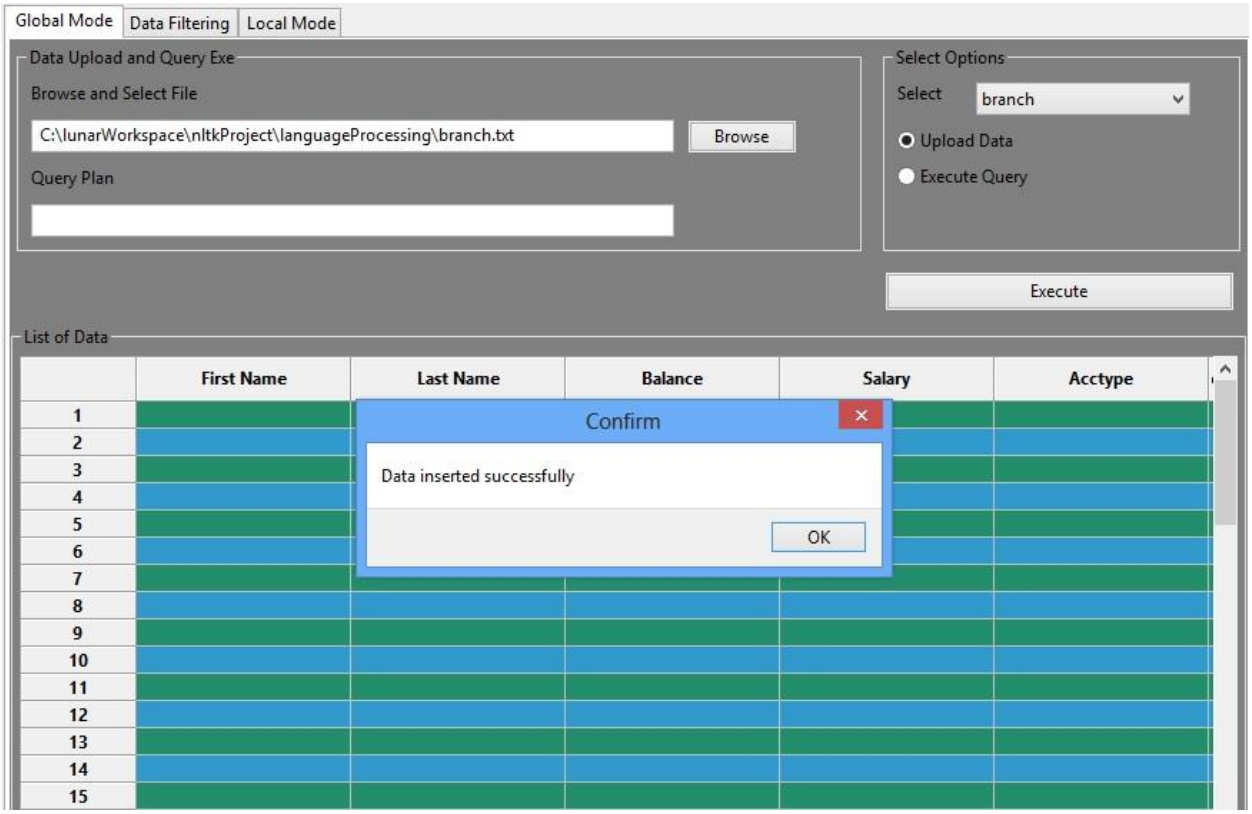
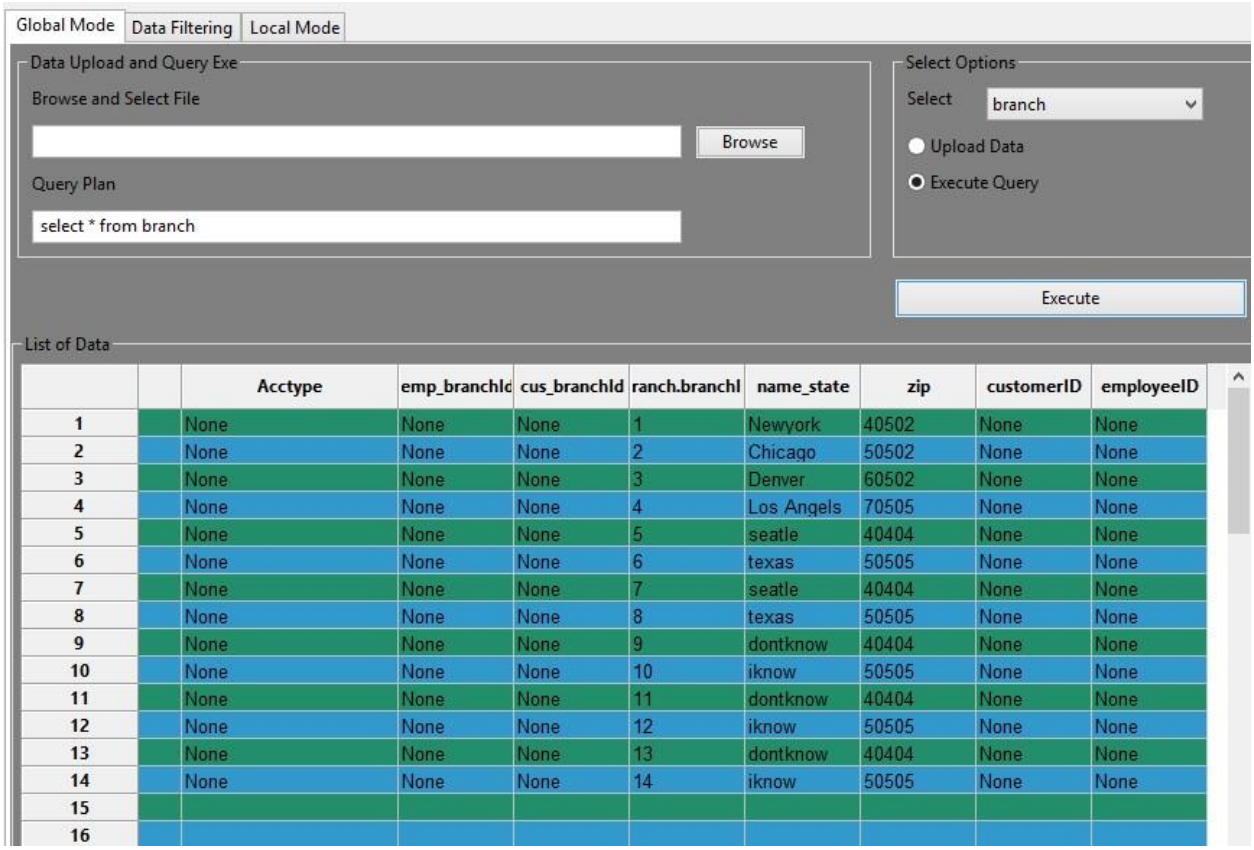


Fig: 12

Above dialog box shows that the attempt for inserting new data using file as a data source into the database is successful.

And in the following image I used the query plan text field to write and run a query. I selected all the data from branch table and the result is displayed in the grid view of the

frame



		Acctype	emp_branchId	cus_branchId	ranch.branchI	name_state	zip	customerID	employeeID
1		None	None	None	1	Newyork	40502	None	None
2		None	None	None	2	Chicago	50502	None	None
3		None	None	None	3	Denver	60502	None	None
4		None	None	None	4	Los Angels	70505	None	None
5		None	None	None	5	seatle	40404	None	None
6		None	None	None	6	texas	50505	None	None
7		None	None	None	7	seatle	40404	None	None
8		None	None	None	8	texas	50505	None	None
9		None	None	None	9	dontknow	40404	None	None
10		None	None	None	10	iknow	50505	None	None
11		None	None	None	11	dontknow	40404	None	None
12		None	None	None	12	iknow	50505	None	None
13		None	None	None	13	dontknow	40404	None	None
14		None	None	None	14	iknow	50505	None	None
15									
16									

Fig: 14

This grid view is attached to the frame container to display the data obtained from the select query operation.

Data Filtering Pane:

This following frame has been designed to provide user a friendly interface to delete specific data from database without creating or forming any query. However the control logic beneath the user interface is remained unimplemented. So this option can be a future extension.

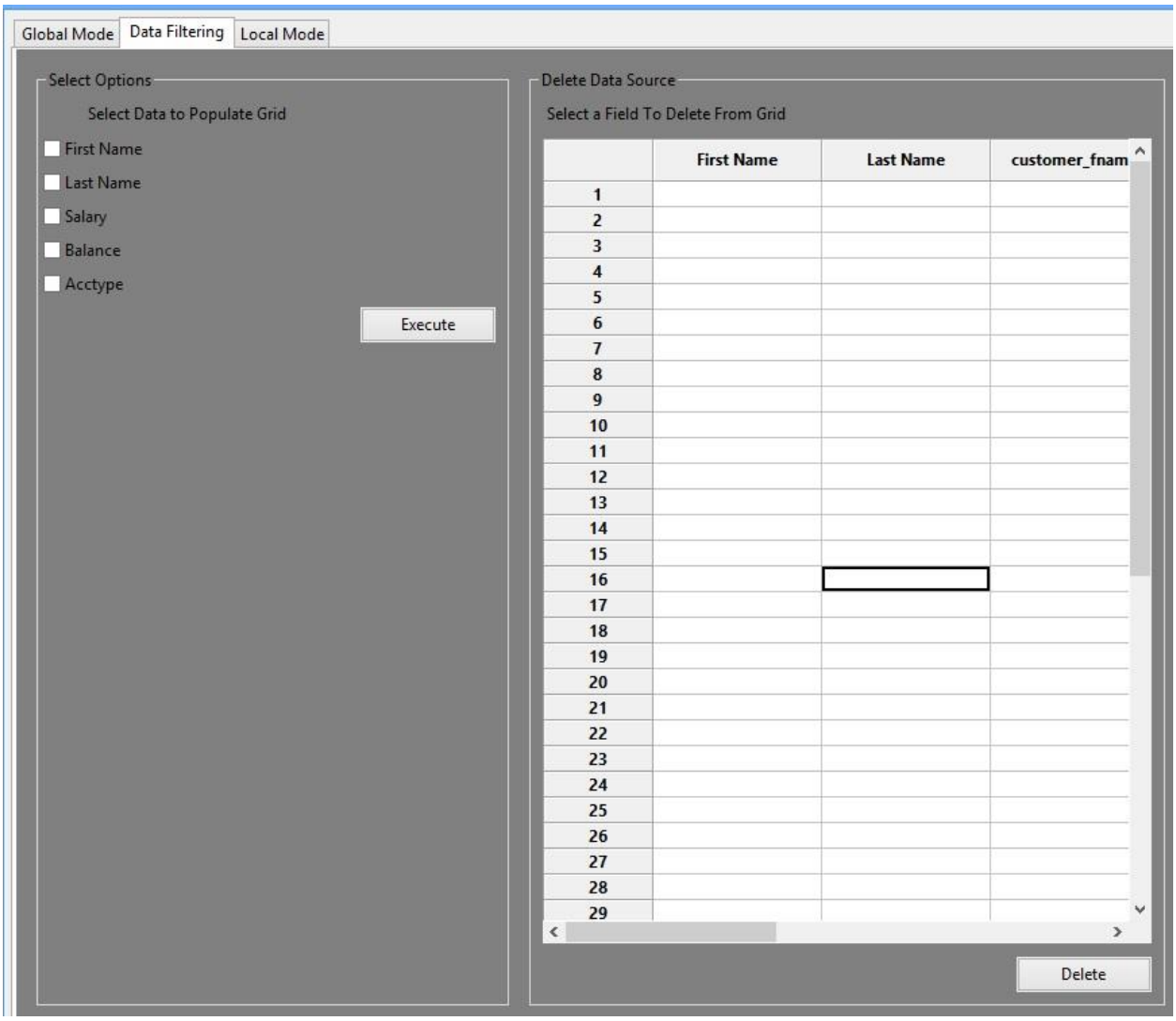


Fig: 15

Control Logic:

I used the following code snippet in python to create four different threads and made the main thread to wait upon these four branches until they finished and return the result. I used one connection to connect with the database and sync the four threads on this single connection object. Other option could have been create four different connections for four different threads however that would have been an overhead and made my user interface responding slowly. Nevertheless if we have large amount of data then four connections may be viable choice.

```
class myThread (threading.Thread):
    def __init__(self, threadID, name, counter,con,query):
        threading.Thread.__init__(self)
        self.threadID = threadID
```

```
        self.name = name
        self.counter = counter
        self.query = query
        self.con =con
        self.error_mes = None
        self.dataObj = dataEncaptulation()
    def run(self):
        threadLock.acquire()
        print "Starting " + self.name
        # Get lock to synchronize threads
        operator = self.con.cursor()

#    try:
        try:
            operator.execute(self.query)
            data = operator.fetchall()
            success = 1
            print('execution done')
        except mysql.connector.Error as err:
            self.error_mes = "Something went wrong: {}".format(err)
            success = 0
            #self.con.rollback()
            #self.con.close()
            operator.close()
            del operator
            threadLock.release()
            return
        except MySQLdb.Error, e:
            #self.con.rollback()
            #self.con.close()
            operator.close()
            del operator
            self.error_mes = "MySQL Error: %s" + str(e)
            threadLock.release()
            #print "MySQL Error: %s" % str(e)
            return

        for row in data:
            tempList= []
            for col in row:
                tempList.append(col)
            self.dataObj.data.append(tempList)
        operator.close()
        del operator
        del data
        #return self.dataObj

        #print_time(self.name, self.counter, 3)
        # Free lock to release next thread
        threadLock.release()

def print_time(threadName, delay, counter):
    while counter:
        time.sleep(delay)
        print "%s: %s" % (threadName, time.ctime(time.time()))
```

```
        counter -= 1

threadLock = threading.Lock()
threads = []

# Create new threads
def start_processing_thread(con,query):
    thread1 = myThread(1, "Thread-1", 1,con,query[0])
    thread2 = myThread(2, "Thread-2", 2,con,query[1])
    thread3 = myThread(3, 'Thread-3', 3,con,query[2])
    thread4 = myThread(4, 'Thread-4', 4,con,query[3])
```

so the main logic is to create four different copies of the submitted query and create four different threads to service these four queries. Get the result from four different threads and combine the results to get the desired result.

Following image shows the main logic of map reduce that we want to implement. We took the query map them into four different branches and wait for the result. When all the branches are done with their works then we combine the result of different branches.

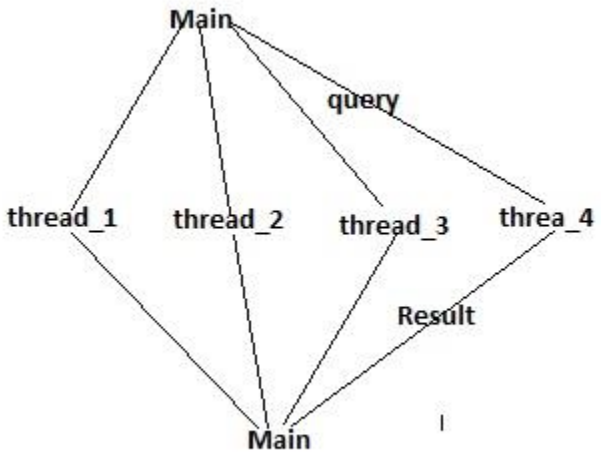


Fig: 17

For instance , I ran the following query
“Select max(salary) from employee” and the four threads returned the following results,

```
 Resultant list:[[Decimal('100000.00')], [Decimal('110000.00')],
[Decimal('65000.00'), [Decimal('90000.00')]],

<myThread(Thread-1, stopped 10592)>
[Decimal('100000.00')]]
<myThread(Thread-2, stopped 6376)>
[[Decimal('110000.00')]]
<myThread(Thread-3, stopped 5504)>
[[Decimal('65000.00')]]
<myThread(Thread-4, stopped 14020)>
```

```
[[Decimal('90000.00')]]
Exiting main Thread
```

As you can see each thread returns individual list. Then four list combined together to generate single large list. Later this list of data has been used to get the desired result.

Error Handling:

This is one of most significant part. Basically this solution has a many source of producing bugs. I tried to handle as many of them as possible. To make this application completely bug free we need to run rigorous software testing procedure on it. I am going to show some of the bugs those I handled effectively.

Duplicate Data Entry Error:

if I try to enter data with duplicate private key following error message will show up

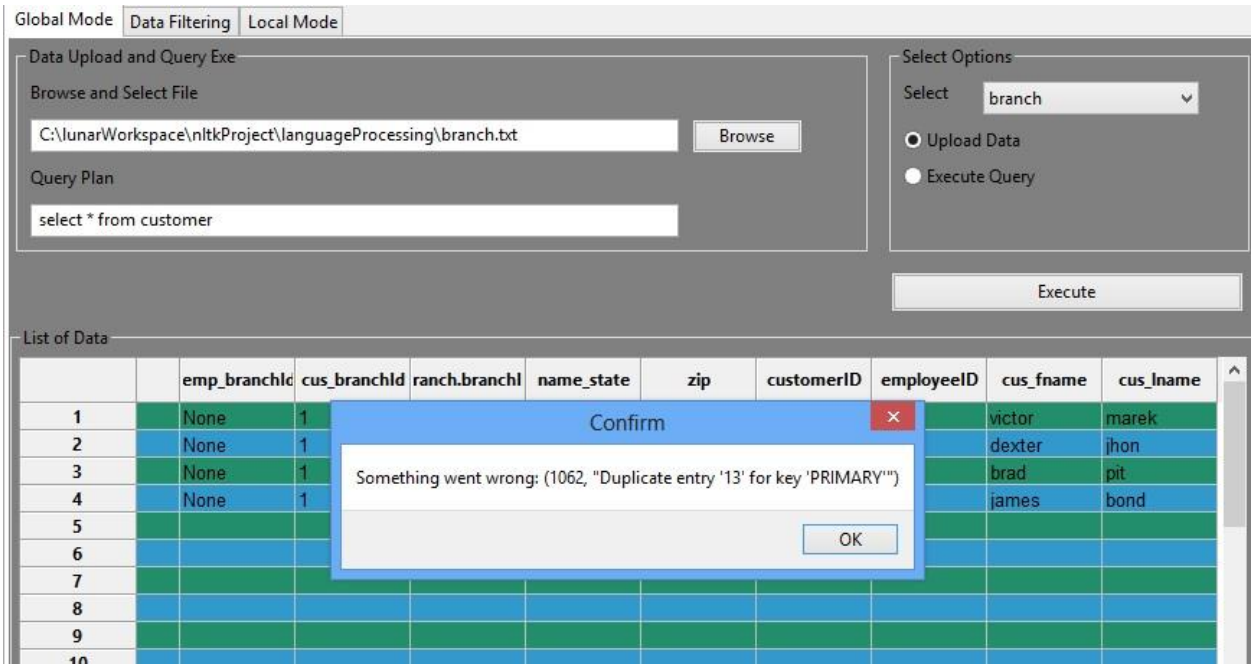


Fig: 17

As you can see the file branch.txt contains some data that has conflict with the data present in the database.

Data Format Error:

If I select a wrong table to load data then it will show following error message.

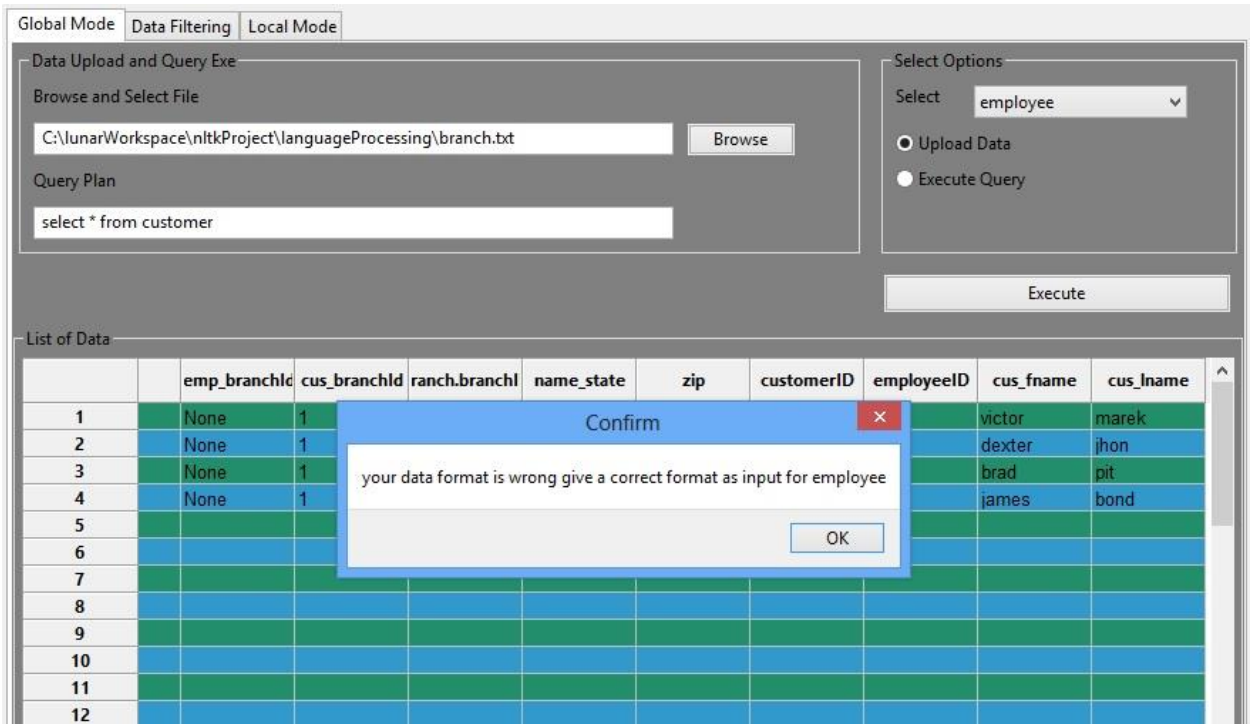


Fig: 18

I have a data file for branch table and I selected the employee table to load information certainly there is a data formatting error.

Query Syntax Error:

I used the exception handling of python MySQLdb module to catch the sql syntactical error generated by the sql engine and returned that message to give the user the required warning
As we can see below,

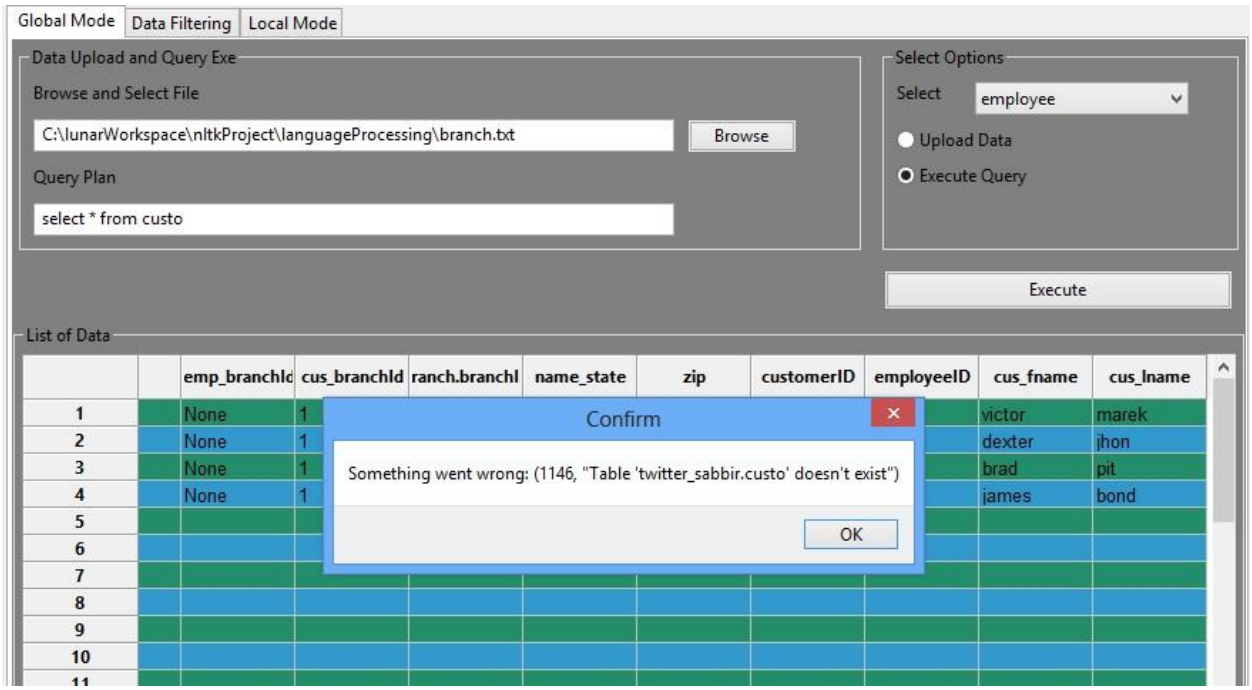


Fig: 19

The error message in above dialog showed that the corresponding table mentioned in the query actually does not exist in the database.

Above three errors are major source of error in local mode.

Improper Table Selection:

Selecting an attribute which is not belong to a table of interest will raise an error like the error message generated below,

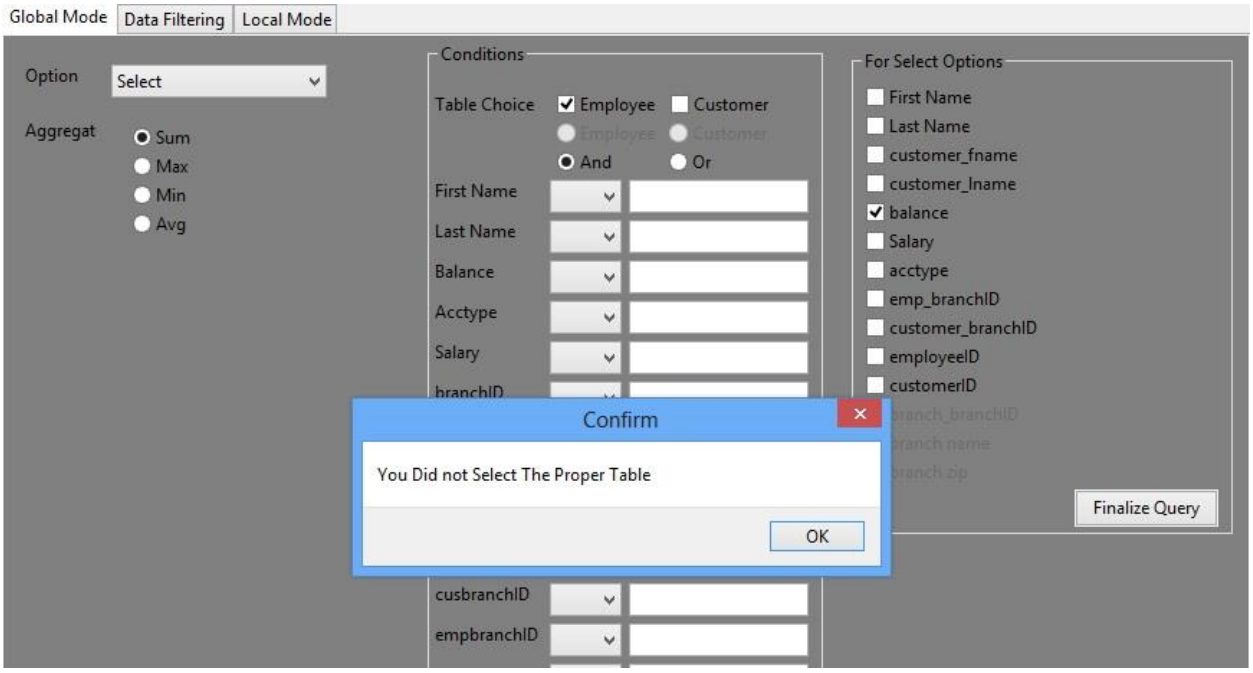


Fig: 20

I have selected the balance attribute for select field which is not a part of employee table so it raised an error.

Attribute Selection Control:

In some cases error has been handled by controlling the selection capacity. Such as in case of aggregate operation user can only select balance or salary attribute like it is marked below with circle,

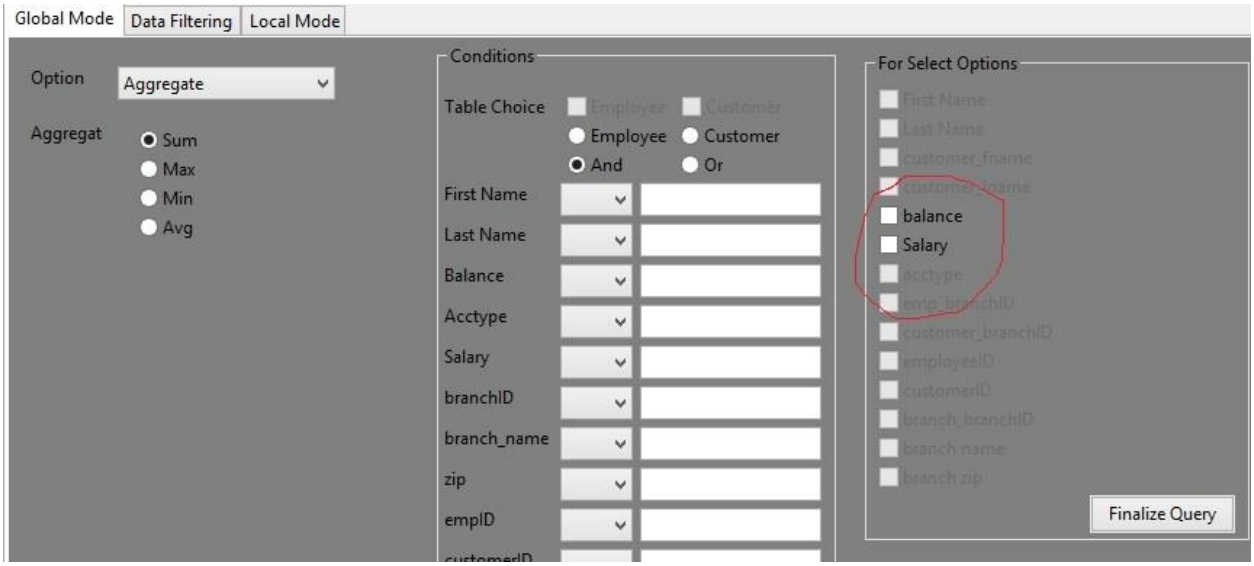


Fig: 21

Before you close the main application window, it will give you the following warning message,

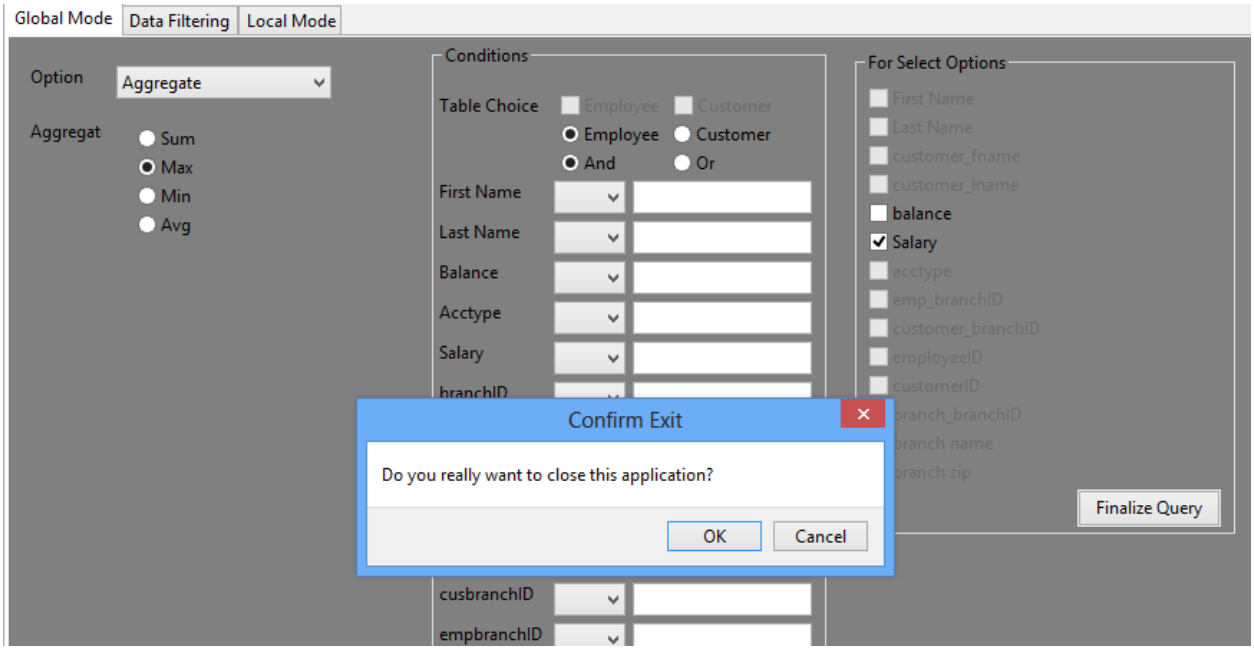


Fig: 22

As you can see it is asking for the confirmation to exit.

So the code for application is,

```
import wx
import Image
#########################################################################
import ctypes  # An included library with Python install.
import wx.grid
import re
from dbHW1 import fetchData, start_processing_thread
from dbHW1 import insertData
from dbHW1 import getConnector
import os
import random
from msilib.schema import RadioButton
from languageProcessing import dbHW1
class gridTable(wx.grid.PyGridTableBase):
    def __init__(self,rows,cols):
        wx.grid.PyGridTableBase.__init__(self)
        self.rows = rows
        self.cols = cols
        self.odd = wx.grid.GridCellAttr()
        self.data = {}#[[]*self.cols for i in xrange(self.rows)]
        self.rowLabels = ['First Name','Last
Name','Balance','Salary','Acctype','emp_branchId','cus_branchId', 'branch.branchID',
'name_state','zip','customerID','employeeID','cus_fname','cus_lname']
        self.odd.SetBackgroundColour("sky blue")
```

```
        self.odd.SetFont(wx.Font(10,wx.SWISS,wx.NORMAL,wx.NORMAL))
        self.even = wx.grid.GridCellAttr()
        self.even.SetBackgroundColour("sea green")
        self.even.SetFont(wx.Font(10,wx.SWISS,wx.NORMAL,wx.NORMAL))
        return
    def IsEmptyCell(self,row,col):
        return self.data.get((row,col))
    def GetColLabelValue(self,row):
        return self.rowLabels[row]
    def SetCellSize(self,row,col,width,height):
        self.Set
        return
    def GetValue(self,row,col):
        value = self.data.get((row,col))
        if(value is not None):
            return value
        else:
            return ''
    def GetNumberRows(self):
        return self.rows
    def GetNumberCols(self):
        return self.cols
    def SetValue(self,row,col,value):
        self.data[(row,col)] = value
        return
    def DeleteRows(self,row,numRows =1):

        if(row <= self.GetNumberRows()):
            print(str(row) + " "+str(self.GetNumberRows()))
            #del self.data[(row,1)]
            return True
        else:
            return False
    def GetAttr(self,row,col,kind):
        attr = [self.even, self.odd][row%2]
        attr.IncRef()
        return attr
    def AppendRows(self,numRows = 1):
        return (self.GetNumberRows() + numRows) <=100
class createGrid():
    headerList = ['First Name','Last
Name','customer_fname','customer_lname','Balance','Salary','Acctype','Employee
BranchId','Customer
BranchId','branch.branchID','branch_name','zip','customerID','employeeID','cus_fname'
,'cus_lname']
    def __init__(self,container,rows,cols):
        self.grid = wx.grid.Grid(container)
        self.grid.CreateGrid(rows,cols)
        self.grid.SetColSize(0,125)
        self.grid.SetColSize(1,125)
        self.grid.SetColSize(2,125)
        self.grid.SetColSize(3,125)
        self.grid.SetColSize(4,125)
        self.grid.SetColSize(5,125)
        self.grid.SetColSize(6,125)
```

```
        '''self.grid.SetColSize(8,125)
        self.grid.SetColSize(9,125)
        self.grid.SetColSize(10,125)'''
        for i in range(0,cols):
            self.grid.SetColLabelValue(i,self.headerList[i])
        return
    def getGrid(self):
        return  self.grid
    def addItem(self,rowNumber,itemList):
        for i in range(len(itemList)):
            self.grid.SetCellValue(rowNumber,i,"%s" % (itemList[i]))
        return

def Mbox(title, text, style):
    ctypes.windll.user32.MessageBoxA(0, text, title, style)

mainFrame = None
#conn = None
class TabPanel(wx.Panel):

        """
       This will be the first notebook tab
       """
        ischecked_finance = False
        ischecked_gen=False
        ischecked_tech=False
        ischecked_hr=False
    #---------------------------------------------------------------------
        def __init__(self, parent,page_number,get_scope):
            wx.Panel.__init__(self, parent=parent, id=wx.ID_ANY)
            self.parent=parent
            self.query = ""
            self.querys = [None]*4
            self.option=""
            self.dict_list ={}
            self.item_index={}
            self.first_name_cond =''
            self.last_name_cond =''
            self.salary_cond=''
            self.balance_cond=''
            self.acctyp_cond=''
            self.branch_id_cond =''
            self.branch_name_cond=''
            self.branch_zip_cond = ''
            self.emp_id_cond =''
            self.cus_id_cond =''
            self.cus_branch_id_cond=''
            self.emp_branch_id_cond=''
            self.get_scope = get_scope
            if(page_number == 1):
                if(self.get_scope =='global'):
                    self.tab_page_one(get_scope)
                else:
                    self.tab_page_three()
            elif(page_number == 2):
```

```
                self.tab_page_two()
            elif(page_number == 3):
                if(self.get_scope =='global'):
                    self.tab_page_three()
                else:
                    self.tab_page_one(get_scope)
            '''elif(page_number == 4):
                self.tab_page_four()
            else:
                self.tab_page_five()'''
            return
        # insert butten button at condition check pane
        def insert_execute(self,event):
            if(self.get_scope =='local'):
                Mbox('Warning', 'You can form query in this mode',
style=wx.FONTFAMILY_DEFAULT)
                return
            conn = getConnector(machine = 'localhost',username = 'root',passwd =
'ztwitasd4',port =3306,databaseName='twitter_sabbir')
            dataOb = None
            print(self.query)
            if(self.cb1.GetValue() == 'branch'):
                dataOb =  fetchData( con=conn,sqlquery=self.query)
            else:
                dataOb = start_processing_thread(conn,self.querys)
            # if(isinstance(dataOb,dataEncaptulation)):

            if(dataOb == None):
                self.showDialog("No data Available to Display as None")
                return
            if(isinstance(dataOb,str) == True):
                self.showDialog(dataOb)
                return
            if(len(dataOb.data) == 0 ):
                self.showDialog("No data Available to Display as Zero")
                return
            #self.list_item.Clear()
            #self.list_item.Append(self.list_items[0])
            #auth_list = self.parent.objectToWork.data[0][2:]
            row_number = 0
            self.init_gridI()
            if(self.cb1.GetValue() == 'Select' or self.cb1.GetValue() == 'branch'):
                for data in dataOb.data:
                #zipper = zip(list_list,data)
                #tempStr = ""
                #temp_list_op = data[7:]
                #if(self.matchAccess(auth_list, temp_list_op) ==False):
                #   continue
                    col_number = 0
                    for (key,value) in self.dict_list.items():
                        if(value != 0):
                            print('key = '  + key + 'value = ' + str(value))
                            print('item index = ' + str(self.item_index[key]))
                            self.tableI.SetValue(row_number,value -
1,data[self.item_index[key]])
```

```
                        else:
                            self.tableI.SetValue(row_number,value - 1,'None')
                        col_number += 1
                    row_number += 1
                #self.list_item.Append(tempStr)
                # self.list_item.Append(temp_list)
                self.gridI.ForceRefresh()
            else:
                operations = ''
                tempStr = ''
                if(self.get_scope =='local'):
                    tempStr = self.query
                else:
                    tempStr = self.querys[0]
                resultant_string =
self.getResult(tempStr,dataOb.data,self.comp_select,self.dict_list,self.item_index)
                if(resultant_string == ''):
                    self.showDialog('No Result to Display')
                else:
                    #Mbox('Result',resultant_string,style=wx.FONTFAMILY_DEFAULT)
                    self.showDialog(resultant_string)
            return
        def OnSelect(self,e):
            strings = e.GetString()
            self.option =strings
            #Mbox('Waring',"There is no User of that Name", wx.FONTFAMILY_DEFAULT)
            print("The event is executing at least")
            if(strings == 'Select'):
                self.check_emp_Id.SetValue(False)
                self.check_emp_Id.Enable(True)
                self.check_cus_Id.SetValue(False)
                self.check_cus_Id.Enable(True)
                self.check_employee.Enable(True)
                self.check_customer.Enable(True)
                self.check_salary.Enable(True)
                self.check_balance.Enable(True)
                self.check_first_name.Enable(True)
                self.check_last_name.Enable(True)
                self.check_acctype.Enable(True)
                self.check_emp_branchId.Enable(True)
                self.check_cus_branchId.Enable(True)
                self.check_branch_id_.SetValue(False)
                self.check_branch_id_.Enable(False)
                self.check_branch_name_.SetValue(False)
                self.check_branch_name_.Enable(False)
                self.check_branch_zip.SetValue(False)
                self.check_branch_zip.Enable(False)
                self.check_cus_first_name.SetValue(False)
                self.check_cus_first_name.Enable(True)
                self.check_cus_last_name.SetValue(False)
                self.check_cus_last_name.Enable(True)

            elif(strings == 'Aggregate'):
                self.check_salary.SetValue(False)
                self.check_salary.Enable(True)
```

```
            self.check_balance.SetValue(False)
            self.check_balance.Enable(True)
            self.check_employee.SetValue(False)
            self.check_employee.Enable(False)
            self.check_customer.SetValue(False)
            self.check_customer.Enable(False)
            self.check_first_name.SetValue(False)
            self.check_first_name.Enable(False)
            self.check_last_name.SetValue(False)
            self.check_last_name.Enable(False)
            self.check_acctype.SetValue(False)
            self.check_acctype.Enable(False)
            self.check_emp_branchId.SetValue(False)
            self.check_emp_branchId.Enable(False)
            self.check_cus_branchId.SetValue(False)
            self.check_cus_branchId.Enable(False)
            self.check_branch_id_.SetValue(False)
            self.check_branch_id_.Enable(False)
            self.check_branch_name_.SetValue(False)
            self.check_branch_name_.Enable(False)
            self.check_branch_zip.SetValue(False)
            self.check_branch_zip.Enable(False)
            self.check_emp_Id.SetValue(False)
            self.check_emp_Id.Enable(False)
            self.check_cus_Id.SetValue(False)
            self.check_cus_Id.Enable(False)
            self.check_cus_first_name.SetValue(False)
            self.check_cus_first_name.Enable(False)
            self.check_cus_last_name.SetValue(False)
            self.check_cus_last_name.Enable(False)
        else:
            self.check_emp_Id.SetValue(False)
            self.check_emp_Id.Enable(False)
            self.check_cus_Id.SetValue(False)
            self.check_cus_Id.Enable(False)
            self.check_emp_branchId.Enable(False)
            self.check_cus_branchId.Enable(False)
            self.check_salary.SetValue(False)
            self.check_salary.Enable(False)
            self.check_balance.SetValue(False)
            self.check_balance.Enable(False)
            self.check_employee.SetValue(False)
            self.check_employee.Enable(False)
            self.check_customer.SetValue(False)
            self.check_customer.Enable(False)
            self.check_first_name.SetValue(False)
            self.check_first_name.Enable(False)
            self.check_last_name.SetValue(False)
            self.check_last_name.Enable(False)
            self.check_acctype.SetValue(False)
            self.check_acctype.Enable(False)
            self.check_cus_first_name.SetValue(False)
            self.check_cus_first_name.Enable(False)
            self.check_cus_last_name.SetValue(False)
            self.check_cus_last_name.Enable(False)
```

```
                self.check_emp_branchId.SetValue(False)
                self.check_emp_branchId.Enable(False)
                self.check_cus_branchId.SetValue(False)
                self.check_cus_branchId.Enable(False)
                self.check_branch_id_.Enable(True)
                self.check_branch_name_.Enable(True)
                self.check_branch_zip.Enable(True)
            if(strings == 'Aggregate'):
                self.check_gen_emp.Enable(True)
                self.check_hr_cus.Enable(True)
            else:
                self.check_gen_emp.Enable(False)
                self.check_hr_cus.Enable(False)
            return
        def Show_employee(self,event):
            ob = event.GetEventObject()
            self.ischecked_finance = ob.GetValue()
            return
        def Show_Gen_Emp(self,event):
            ob = event.GetEventObject()
            self.ischecked_gen = ob.GetValue()
            return
        def Show_Customer(self,event):
            ob = event.GetEventObject()
            self.ischecked_tech = ob.GetValue()
            return
        def Show_Hr_Cus(self,event):
            ob = event.GetEventObject()
            self.ischecked_hr = ob.GetValue()
            return
        def select_execute(self,event):

            if(self.get_scope =='local'):
                Mbox('Warning', 'You can not form query in this mode',
style=wx.FONTFAMILY_DEFAULT)
                return
            fn = self.check_first_name.GetValue()
            ln = self.check_last_name.GetValue()
            acctype = self.check_acctype.GetValue()
            balance = self.check_balance.GetValue()
            salary = self.check_salary.GetValue()
            emp_branch = self.check_emp_branchId.GetValue()
            cus_branch = self.check_cus_branchId.GetValue()
            branch_id = self.check_branch_id_.GetValue()
            branch_name = self.check_branch_name_.GetValue()
            branch_zip = self.check_branch_zip.GetValue()
            customer_id = self.check_cus_Id.GetValue()
            employee_id = self.check_emp_Id.GetValue()
            cfn = self.check_cus_first_name.GetValue()
            cln = self.check_cus_last_name.GetValue()
            self.query='select '
            if(self.option== '' or len(self.option) == 0):
                self.showDialog("Option field is Empty")
                return
```

```
            aggregate=""
            if(self.option == 'Aggregate'):
                if(self.agg_choice_1.GetValue() == True):
                    aggregate= 'SUM'
                elif(self.agg_choice_2.GetValue() ==True):
                    aggregate = 'MAX'
                elif(self.agg_choice_3.GetValue() == True):
                    aggregate = 'MIN'
                else:
                    aggregate ='AVG'
            if(self.option == 'Select'):
                if(self.check_employee.GetValue() == False and
self.check_customer.GetValue() == False):
                    self.showDialog("Select at Least one Table to Work on")
                    return
            elif(self.option =='Aggregate'):
                    if(self.check_gen_emp.GetValue() == False and
self.check_hr_cus.GetValue() == False):
                        self.showDialog("Select one Table to Work on")
                        return

            if(fn == False and ln == False and salary == False and balance == False
and emp_branch == False and cus_branch == False and acctype == False and branch_id ==
False and branch_name == False and branch_zip == False and customer_id == False and
employee_id == False and cfn ==False and cln == False):
                self.showDialog("Select at Least one Field")
                return
            #compartment = self.check_compartments.GetValue()
            list_list = []
            self.dict_list =
{'emplname':0,'empfname':0,'balance':0,'salary':0,'acctype':0,'employee.branchID':0,'
customer.branchID':0,'branch.branchID':0,'name_state':0,'zip':0,'customerID':0,'emplo
yeeID':0,'first_name':0,'last_name':0}
            self.item_index =
{'emplname':0,'empfname':0,'balance':0,'salary':0,'acctype':0,'employee.branchID':0,'
customer.branchID':0,'branch.branchID':0,'name_state':0,'zip':0,'customerID':0,'emplo
yeeID':0,'first_name':0,'last_name':0}
            i = 0
            for_indexing = 0
            if(ln == True):
                if(self.check_employee.GetValue() == True ):
                    self.query += 'emplname, '
                    self.dict_list['emplname'] = 2
                    self.item_index['emplname'] = for_indexing
                    for_indexing += 1
                else:
                    self.showDialog('Select Proper Table')
                    return
                list_list.append('firstName')
                #dict_list[2] = 1
            if(branch_id == True):
                self.query += 'branch.branchID, '
                self.dict_list['branch.branchID'] = 8
                self.item_index['branch.branchID'] = for_indexing
                for_indexing += 1
```

```
if(branch_name == True):
    self.query += 'name_state, '
    self.dict_list['name_state'] = 9
    self.item_index['name_state'] = for_indexing
    for_indexing += 1
if(branch_zip == True):
    self.query += 'zip, '
    self.dict_list['zip'] = 10
    self.item_index['zip'] = for_indexing
    for_indexing += 1

if(fn==True):
    if(self.check_employee.GetValue() ==True ):
        self.query += 'empfname, '
        self.dict_list['empfname'] = 1
        self.item_index['empfname'] = for_indexing
        for_indexing += 1
    else:
        self.showDialog('Select Proper Table')
        return
if(cfn == True):
    if(self.check_customer.GetValue() ==True ):
        self.query += 'first_name, '
        self.dict_list['first_name'] = 13
        self.item_index['first_name'] = for_indexing
        for_indexing += 1
    else:
        self.showDialog('Select Proper Table for This Attribute')
        return
if(cln == True):
    if(self.check_customer.GetValue() ==True ):
        self.query += 'last_name, '
        self.dict_list['last_name'] = 14
        self.item_index['last_name'] = for_indexing
        for_indexing += 1
    else:
        self.showDialog('Select Proper Table for This Attribute')
        return

if(fn==True):
    if(self.check_employee.GetValue() ==True ):
        self.query += 'empfname, '
        self.dict_list['empfname'] = 1
        self.item_index['empfname'] = for_indexing
        for_indexing += 1
    else:
        self.showDialog('Select Proper Table')
        return
if(customer_id == True):
    if(self.check_customer.GetValue() == True ):
        self.query += 'customerID, '
        self.dict_list['customerID'] = 11
        self.item_index['customerID'] = for_indexing
        for_indexing += 1
    else:
```

```
            self.showDialog('Select Proper Table')
            return
    if(employee_id == True):
        if(self.check_employee.GetValue() ==True ):
            self.query += 'employeeID, '
            self.dict_list['employeeID'] = 12
            self.item_index['employeeID'] = for_indexing
            for_indexing += 1
        else:
            self.showDialog('Select Proper Table')
            return
        #dict_list[3] =  1
    if(salary ==True):
        if(self.option =='Select'):
            if(self.check_employee.GetValue() == True ):
                self.query += 'salary, '
                self.dict_list['salary'] = 4
                self.item_index['salary'] = for_indexing
                for_indexing += 1
            else:
                self.showDialog('You Did not Select the Table')
                return
        else:
            if(self.check_gen_emp.GetValue() == True):
                if(aggregate == 'AVG'):
                    self.query += 'SUM(salary), COUNT(*), '
                else:
                    self.query += aggregate +'(salary), '
                self.dict_list['salary'] = 4
                self.item_index['salary'] = for_indexing
                for_indexing += 1
            else:
                self.showDialog('You did not select proper table')

        list_list.append('age')
        #dict_list[4] =  1
    if(balance == True):
        if(self.option =='Select'):
            if(self.check_customer.GetValue() ==True ):
                self.query += 'balance, '
                self.dict_list['balance'] = 3
                self.item_index['balance'] = for_indexing
                for_indexing += 1
            else:
                self.showDialog('You Did not Select The Proper Table')
                return
        else:
            if(self.check_hr_cus.GetValue() == True):
                if(aggregate == 'AVG'):
                    self.query += 'SUM(balance), COUNT(*), '
                else:
                    self.query += aggregate +'(balance), '
                self.dict_list['balance'] = 3
                self.item_index['balance'] = for_indexing
                for_indexing += 1
```

```
                else:
                    self.showDialog('You did not select proper table')
            if(acctype == True):
                if(self.check_customer.GetValue() == True):
                    self.query += 'acctype, '
                    self.dict_list['acctype'] = 5
                    self.item_index['acctype'] = for_indexing
                    for_indexing += 1
                else:
                    self.showDialog('you did not Select proper table')
            emp_q = ['']*4
            if(emp_branch ==True):
                if(self.check_employee.GetValue() ==True):
                    #self.query += 'employee.branchID, '
                    emp_q[0] = 'employee_1.branchID, '
                    emp_q[1] = 'employee_2.branchID, '
                    emp_q[2] = 'employee_3.branchID, '
                    emp_q[3] = 'employee_4.branchID, '
                    self.dict_list['employee.branchID'] = 6
                    self.item_index['employee.branchID'] = for_indexing
                    for_indexing += 1
                else:
                    self.showDialog('You did not select the proper Table')
            if(cus_branch ==True):
                if(self.check_customer.GetValue()):
                    #self.query += 'customer.branchID, '
                    emp_q[0] += 'customer_1.branchID '
                    emp_q[1] += 'customer_2.branchID '
                    emp_q[2] += 'customer_3.branchID '
                    emp_q[3] += 'customer_4.branchID '
                    self.dict_list['customer.branchID'] = 7
                    self.item_index['customer.branchID'] = for_indexing
                    for_indexing += 1
                else:
                    self.showDialog('You did not select proper table')
                list_list.append('salary')
                #dict_list[5] =  1
            if(emp_q[0] == ''):
                self.query = self.query.rstrip()
                self.query = self.query.rstrip(',')
                split_list = self.query.split(' ')
                if(len(split_list) == 1):
                    self.showDialog("Your attribute Selection is inConsistent, Try to
rebuild query again")
                    return
            else:
                if(emp_q[0][len(emp_q[0]) - 2] ==','):
                    for i in xrange(len(emp_q)):
                        emp_q[i] = emp_q[i].split(',')[0]
            self.comp_select  = self.query
            print('query is:' + self.query)
            from_list =' from'
            from_list_arr = [None]*4
            #////////////////////////////from list
////////////////////////////////////////////////
```

```
            if(self.check_employee.GetValue() == True or
self.check_gen_emp.GetValue() ==True):
                if(self.get_scope == 'local'):
                    from_list += '
employee_'+str(self.parent.objectToWork.data[3])+','
                else:
                    for i in xrange(1,5):
                        if(emp_q[0] !=  ''):
                            from_list_arr[i-1] = emp_q[i-1] + from_list + '
employee_'+str(i)+','
                        else:
                            from_list_arr[i-1] =  from_list + ' employee_'+str(i)+','
                    emp_q[0] = ''
            if(self.check_customer.GetValue() == True or self.check_hr_cus.GetValue()
==True):
                if(self.get_scope =='local'):
                    from_list += '
customer_'+str(self.parent.objectToWork.data[3])+','
                else:
                    for i in xrange(1,5):
                        if(from_list_arr[i-1] != None):
                            if(emp_q[0] != ''):
                                from_list_arr[i-1] =emp_q[i-1]+ from_list_arr[i-1] +'
customer_'+str(i)+','
                            else:
                                from_list_arr[i-1] = from_list_arr[i-1] +'
customer_'+str(i)+','
                        else:
                            if(emp_q[0] != ''):
                                from_list_arr[i-1] = emp_q[i-1]+ from_list +'
customer_'+str(i)+','
                            else:
                                from_list_arr[i-1] = from_list +'
customer_'+str(i)+','
            if(self.cb1.GetValue() =='branch'):
                from_list +=' branch '
            # ////////////////////////////end of from list
//////////////////////////////////////
            if(self.cb1.GetValue() =='branch'):
                from_list = from_list.rstrip(',')
            else:
                #print(str(len(from_list_arr)))
                for each in xrange(0,len(from_list_arr)):
                    from_list_arr[each] = from_list_arr[each].rstrip(',')
                    print(from_list_arr[each])

            if(self.cb1.GetValue() =='branch'):
                self.query += from_list
            else:
                for i in xrange(1,5):
                    self.querys[i-1] = self.query + from_list_arr[i-1]
            cond_list = ' where '
            if(self.first_name_cond != '' and (self.check_employee.GetValue() == True
or self.check_gen_emp.GetValue() ==True)):
                if(self.text_first_name.GetValue() != ''):
```

```
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list + "empfname "+self.first_name_cond+" '"+
self.text_first_name.GetValue()+"' "
            if(self.text_last_name !='' and (self.check_employee.GetValue() == True
or self.check_gen_emp.GetValue() ==True)):
                if(self.text_last_name.GetValue() !=''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list +"emplname "+ self.last_name_cond+"
'"+self.text_last_name.GetValue()+"' "
            if(self.balance_cond != '' and (self.check_customer.GetValue() == True or
self.check_hr_cus.GetValue() ==True)):
                if(self.text_balance.GetValue()!=''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+"balance " +self.balance_cond+" '"+
self.text_balance.GetValue()+"' "
            if(self.salary_cond !='' and (self.check_employee.GetValue() == True or
self.check_gen_emp.GetValue() ==True)):
                if(self.text_salary.GetValue()!= ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list = cond_list+"salary "+self.salary_cond+" '"+
self.text_salary.GetValue()+"' "
            if(self.acctyp_cond !='' and (self.check_customer.GetValue() == True or
self.check_hr_cus.GetValue() ==True)):
                if(self.text_acctype.GetValue()!=''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'acctype '+ self.acctyp_cond +"
'"+self.text_acctype.GetValue() +"' "
            if(self.branch_id_cond != '' and self.cb1.GetValue() == 'branch'):
                if(self.text_branch_id.GetValue()  != '' ):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'branch.branchID '+ self.branch_id_cond +"
'"+self.text_branch_id.GetValue() +"' "
            if(self.branch_name_cond != '' and self.cb1.GetValue() == 'branch'):
                if(self.text_branch_name.GetValue()  != ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'name_state '+ self.branch_name_cond +"
'"+self.text_branch_name.GetValue() +"' "
            if(self.branch_zip_cond != '' and self.cb1.GetValue() == 'branch'):
                if(self.text_branch_zip.GetValue()  != ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'zip '+ self.branch_zip_cond +"
'"+self.text_branch_zip.GetValue() +"' "
            if(self.emp_id_cond != '' and (self.check_employee.GetValue() == True or
self.check_gen_emp.GetValue() ==True)):
                if(self.text_emp_id.GetValue()  != ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
```

```
                    cond_list =cond_list+'employeeID '+ self.emp_id_cond +"
'"+self.text_emp_id.GetValue() +"' "
            if(self.cus_id_cond != '' and (self.check_customer.GetValue() == True or
self.check_hr_cus.GetValue() ==True)):
                if(self.text_cus_id.GetValue()  != ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'customerID '+ self.cus_id_cond +"
'"+self.text_cus_id.GetValue() +"' "
            if(self.cus_branch_id_cond != '' and (self.check_customer.GetValue() ==
True or self.check_hr_cus.GetValue() ==True)):
                if(self.text_cus_branch_id.GetValue()  != ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'customer_.branchID '+
self.cus_branch_id_cond +" '"+self.text_cus_branch_id.GetValue() +"' "
            if(self.emp_branch_id_cond != '' and (self.check_employee.GetValue() ==
True or self.check_gen_emp.GetValue() ==True)):
                if(self.text_emp_branch_id.GetValue()  != ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'employee_.branchID '+
self.emp_branch_id_cond +" '"+self.text_emp_branch_id.GetValue() +"' "
            if(self.text_cus_first_name_cond.GetValue() != '' and
(self.check_customer.GetValue() == True or self.check_hr_cus.GetValue() ==True)):
                if(self.text_cus_first_name.GetValue()  != ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'first_name '+
self.text_cus_first_name_cond.GetValue() +" '"+self.text_cus_first_name.GetValue()
+"' "
            if(self.text_cus_last_name_cond.GetValue() != '' and
(self.check_customer.GetValue() == True or self.check_hr_cus.GetValue() ==True)):
                if(self.text_cus_last_name.GetValue()  != ''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'last_name '+
self.text_cus_last_name_cond.GetValue() +" '"+self.text_cus_last_name.GetValue() +"'
"
            '''if(self.text_br !=''):
                if(self.text_acctype.GetValue()!=''):
                    if(len(cond_list.strip().split(' ')) > 1):
                        cond_list += 'and '
                    cond_list =cond_list+'acctype '+ self.acctyp_cond +"
'"+self.text_acctype.GetValue() +"' "'''
            cond_list =cond_list.rstrip()
            temp_list = cond_list.strip().split(' ')
            if(len(temp_list ) > 1):
                if(self.cb1.GetValue() =='branch'):
                    self.query =self.query + ' ' + cond_list
                else:
                    for i in xrange(0,4):
                        temp_cond_list =
cond_list.replace('customer_.','customer_'+str(i+1)+'.')
                        print(temp_cond_list)
```

```
                        temp_cond_list =
temp_cond_list.replace('employee_.','employee_'+str(i+1)+'.')
                        print(temp_cond_list)
                        self.querys[i] = self.querys[i] + ' '+ temp_cond_list
            if(self.cb1.GetValue() == 'branch'):
                print('query after from ' + self.query)
            else:
                for each in self.querys:
                    print(each)
            if(self.querys[0] != None):
                temp_str = self.querys[0]

                self.showDialog('your newly formed query \n' + '"' +re.sub('_\d', '',
temp_str, re.I)+'"')
            else:
                self.showDialog('your newly formed query \n' + '"' +self.query+'"')
            '''conn = getConnector(machine = 'localhost',username = 'root',passwd =
'ztwitasd4',port =3306,databaseName='twitter_sabbir')
            dataOb = None
            if(self.get_scope == 'local'):
                dataOb =  fetchData(tableName='information',
con=conn,deptList=self.query,auth= 0)
            else:
                dataOb = start_processing_thread(conn,self.querys)
            if(dataOb == None):
                self.showDialog("No data Available to Display")
                return
            #self.list_item.Clear()
            #self.list_item.Append(self.list_items[0])
            auth_list = self.parent.objectToWork.data[0][2:]
            row_number = 0
            self.init_gridI()
            if(self.cb1.GetValue() == 'Select'):
                for data in dataOb.data:
                #zipper = zip(list_list,data)
                #tempStr = ""
                #temp_list_op = data[7:]
                #if(self.matchAccess(auth_list, temp_list_op) ==False):
                #   continue
                    col_number = 0
                    for (key,value) in dict_list.items():
                        if(value != 0):
                            print('key = '  + key + 'value = ' + str(value))
                            print('item index = ' + str(item_index[key]))
                            self.tableI.SetValue(row_number,value -
1,data[item_index[key]])
                        else:
                            self.tableI.SetValue(row_number,value - 1,'None')
                        col_number += 1
                    row_number += 1
                #self.list_item.Append(tempStr)
                # self.list_item.Append(temp_list)
                self.gridI.ForceRefresh()
            else:
                operations = ''
```

```
                tempStr = ''
                if(self.get_scope =='local'):
                    tempStr = self.query
                else:
                    tempStr = self.querys[0]
                resultant_string =
self.getResult(tempStr,dataOb.data,comp_select,dict_list,item_index)
                self.showDialog(resultant_string)'''

            return
        def get_actual_result(self,list_items,data_list,string):
            if(string != 'minimum'):
                list_result = [[0.0 for i in range(len(data_list))] for j in
range(len(data_list[0]))]
            else:
                list_result = [[999999999.0 for i in range(len(data_list))] for j in
range(len(data_list[0]))]
            rows = 0
            indicator = 0
            for item in data_list:
                columns = 0
                for col in item:
                    if(col != None):
                        indicator += 1
                        list_result[columns][rows]= float(col)
                        columns += 1
                rows += 1
            result = 0
            if(indicator == 0):
                return None
            else:
                return list_result
        def getResult(self,tempStr,data_list,comp_select,dict_list,item_index):
            list_item = comp_select.split(',')
            list_item[0] = list_item[0].split(' ')[1]
            result_string =''
            if(tempStr.lower().find('min') != -1):
                result = self.get_actual_result(list_item,data_list,'minimum')
                if(result == None):
                    self.showDialog('Data base empty')
                    return 'No Result'
                for i in xrange(0,len(result)):
                    result_string = result_string +'Minimum of '+
list_item[i].strip().split('(')[1][:len(list_item[i].strip().split('(')[1])-1]+' '+
str(min(result[i]))+'\n'
            if(tempStr.lower().find('max') != -1):
                result = self.get_actual_result(list_item,data_list,'maximum')
                if(result == None):
                    self.showDialog('Data base empty')
                    return 'No Result'
                #result = self.get_actual_result(list_item,data_list,'minimum')
                for i in xrange(0,len(result)):
                    result_string = result_string +'Maximum of '+
list_item[i].strip().split('(')[1][:len(list_item[i].strip().split('(')[1])-1]+' '+
str(max(result[i]))+'\n'
```

```
            if(tempStr.lower().find('sum') != -1 and tempStr.lower().find('count') ==
-1):
                result = self.get_actual_result(list_item,data_list,'sum')
                if(result == None):
                    self.showDialog('Data base empty')
                    return 'No Result'
                #result = self.get_actual_result(list_item,data_list,'minimum')
                for i in xrange(0,len(result)):
                    result_string = result_string +'Sum of '+
list_item[i].strip().split('(')[1][:len(list_item[i].strip().split('(')[1])-1]+' '+
str(sum(result[i]))+'\n'
            if(tempStr.lower().find('sum') != -1 and tempStr.lower().find('count') !=
-1):
                result = self.get_actual_result(list_item,data_list,'avg')
                if(result == None):
                    self.showDialog('Data base empty')
                    return 'No Result'
                #result = self.get_actual_result(list_item,data_list,'minimum')
                for i in xrange(0,len(result),2):
                    print(list_item[i])
                    print(list_item)
                    if(sum(result[i+1]) != 0):
                        result_string =result_string +'Average of '+
str(list_item[i].strip().split('(')[1][:len(list_item[i].strip().split('(')[1])-1])+'
'+ str(float(sum(result[i]))/sum(result[i+1]))+'\n'

            return result_string
        def init_gridI(self):

            for i in range(0,self.gridI.GetNumberRows()):
                for j in range(0,self.gridI.GetNumberCols()):
                    self.tableI.SetValue(i,j,"")
            return
        def agg_onSelect_sum(self,e):
            return
        def agg_onSelect_max(self,e):
            return
        def agg_onSelect_min(self,e):
            return
        def agg_onSelect_avg(self,e):
            return
        def tab_page_one(self,get_scope):
            gs = wx.GridSizer(1,3,5,15)

            gs_sub = wx.GridSizer(1,2,5,5)
            data = ['Select','Aggregate','branch']
            condition_data = ['=','<>','like','not like','']
            self.execute = wx.Button(self,wx.ID_ANY,'Execute',size=(100,-1))
            #self.execute.Enable(False)
            self.execute.Bind(wx.EVT_BUTTON,self.insert_execute)
            #exec_sizer = wx.BoxSizer(wx.HORIZONTAL)
            #exec_sizer.Add(self.execute,proportion=0,flag=
wx.RIGHT_ALIGN|wx.Top|wx.RIGHT,border=5)
            execute_label = wx.StaticText(self,-1,'       ',size = (50,-1))
            sizer_execute = wx.BoxSizer(wx.HORIZONTAL)
```

```
sizer_execute.Add(execute_label,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT,border=5)

sizer_execute.Add(self.execute,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx.ALIGN_R
IGHT,border=5)
            gs_sub.AddMany([(execute_label,0,wx.EXPAND),(sizer_execute,0,wx.EXPAND)])
            self.cb1 = wx.ComboBox(self,choices = data,size=(150,-1),style =
wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.cb1.Bind(wx.EVT_COMBOBOX,self.OnSelect)
            self.labelCb = wx.StaticText(self,-1,'Option',size=(50,-1))
            label_aggregate = wx.StaticText(self,-1,'Aggregate Options',size=(50,-1))
            sizer_for_option = wx.BoxSizer(wx.HORIZONTAL)
            sizer_for_option_vertical = wx.BoxSizer(wx.VERTICAL)
            sizer_for_option2 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_for_option_choice = wx.BoxSizer(wx.VERTICAL)
            # aggregate function components
            self.agg_choice_1 = wx.RadioButton(self,label='Sum',style=wx.RB_GROUP)
            self.agg_choice_1.Bind(wx.EVT_RADIOBUTTON,self.agg_onSelect_sum)
            self.agg_choice_2 = wx.RadioButton(self,label='Max')
            self.agg_choice_2.Bind(wx.EVT_RADIOBUTTON,self.agg_onSelect_max)
            self.agg_choice_3 = wx.RadioButton(self,label='Min')
            self.agg_choice_3.Bind(wx.EVT_RADIOBUTTON,self.agg_onSelect_min)
            self.agg_choice_4 = wx.RadioButton(self,label='Avg')
            self.agg_choice_4.Bind(wx.EVT_RADIOBUTTON,self.agg_onSelect_avg)

sizer_for_option_choice.Add(self.agg_choice_1,proportion=0,flag=wx.RIGHT|wx.TOP|wx.LE
FT,border = 5)

sizer_for_option_choice.Add(self.agg_choice_2,proportion=0,flag=wx.RIGHT|wx.TOP|wx.LE
FT,border = 5)

sizer_for_option_choice.Add(self.agg_choice_3,proportion=0,flag=wx.RIGHT|wx.TOP|wx.LE
FT,border = 5)

sizer_for_option_choice.Add(self.agg_choice_4,proportion=0,flag=wx.RIGHT|wx.TOP|wx.LE
FT,border = 5)

sizer_for_option2.Add(label_aggregate,proportion=0,flag=wx.RIGHT|wx.TOP|wx.LEFT,borde
r = 10)

sizer_for_option2.Add(sizer_for_option_choice,proportion=0,flag=wx.RIGHT|wx.TOP|wx.LE
FT,border = 10)

sizer_for_option.Add(self.labelCb,proportion=0,flag=wx.RIGHT|wx.TOP|wx.LEFT,border=10
)

sizer_for_option.Add(self.cb1,proportion=0,flag=wx.RIGHT|wx.TOP,border=10)

sizer_for_option_vertical.Add(sizer_for_option,proportion=0,flag=wx.RIGHT|wx.TOP,bord
er=5)

sizer_for_option_vertical.Add(sizer_for_option2,proportion=0,flag=wx.RIGHT|wx.TOP,bor
der=5)
            #////////////////////////////////////////////////// end of aggregate
components ////////////////////
```

```
            self.check_employee = wx.CheckBox(self,label = 'Employee')
            self.check_employee.SetValue(False)
            ''' if(self.parent.objectToWork.data[0][4]==1):
                self.check_employee.Enable(True)
            else:
                self.check_employee.Enable(False)'''
            self.check_customer = wx.CheckBox(self,label='Customer')
            self.check_customer.SetValue(False)
            '''if(self.parent.objectToWork.data[0][3]==1):
                self.check_customer.Enable(True)
            else:
                self.check_customer.Enable(False)'''
            self.check_gen_emp = wx.RadioButton(self,label
='Employee',style=wx.RB_GROUP)
            self.check_gen_emp.SetValue(False)
            '''if(self.parent.objectToWork.data[0][2] ==1):
                self.check_gen_emp.Enable(True)
            else:
                self.check_gen_emp.Enable(False)'''
            self.check_hr_cus = wx.RadioButton(self,label='Customer')
            ''' if(self.parent.objectToWork.data[0][5] ==1):
                self.check_hr_cus.Enable(True)
            else:
                self.check_hr_cus.Enable(False)'''
            self.check_and = wx.RadioButton(self,label='And',style=wx.RB_GROUP)
            self.check_and.SetValue(True)
            self.check_or = wx.RadioButton(self,label='Or')
            self.check_and.Bind(wx.EVT_RADIOBUTTON,self.select_and)
            self.check_or.Bind(wx.EVT_RADIOBUTTON,self.select_or)
            self.check_employee.Bind(wx.EVT_CHECKBOX, self.Show_employee)
            self.check_gen_emp.Bind(wx.EVT_RADIOBUTTON, self.Show_Gen_Emp)
            self.check_customer.Bind(wx.EVT_CHECKBOX, self.Show_Customer)
            self.check_hr_cus.Bind(wx.EVT_RADIOBUTTON, self.Show_Hr_Cus)
            self.check_hr_cus.SetValue(False)
            sizer_check1 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_check2 =wx.BoxSizer(wx.HORIZONTAL)
            sizer_check3 = wx.BoxSizer(wx.HORIZONTAL)
            stat_box_conditions =
wx.StaticBox(self,label='Conditions',size=(300,300))
            sizer_check_ver = wx.StaticBoxSizer(stat_box_conditions, wx.VERTICAL)
            #sizer_check_ver = wx.BoxSizer(wx.VERTICAL)
            label_text_check = wx.StaticText(self,-1,'Table Choice',size=(75,-1))
            sizer_choice_horizontal = wx.BoxSizer(wx.HORIZONTAL)
            sizer_choice_ver = wx.BoxSizer(wx.VERTICAL)

sizer_check1.Add(self.check_employee,proportion=0,flag=wx.EXPAND|wx.RIGHT,border=5)
            sizer_check1.Add(self.check_customer,proportion=0,flag=wx.EXPAND)

sizer_check2.Add(self.check_gen_emp,proportion=0,flag=wx.EXPAND|wx.RIGHT,border=4)
            sizer_check2.Add(self.check_hr_cus,proportion=0,flag=wx.EXPAND)

sizer_check3.Add(self.check_and,proportion=0,flag=wx.EXPAND|wx.RIGHT,border=34)
            sizer_check3.Add(self.check_or,proportion=0,flag=wx.EXPAND)
```

```
sizer_choice_ver.Add(sizer_check1,proportion=0,flag=wx.EXPAND|wx.RIGHT|wx.TOP,border=
5)

sizer_choice_ver.Add(sizer_check2,proportion=0,flag=wx.EXPAND|wx.RIGHT|wx.TOP,border=
5)

sizer_choice_ver.Add(sizer_check3,proportion=0,flag=wx.EXPAND|wx.RIGHT|wx.TOP,border=
5)

sizer_choice_horizontal.Add(label_text_check,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RI
GHT,border=10)

sizer_choice_horizontal.Add(sizer_choice_ver,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RI
GHT,border=5)
            sizer_check_ver.Add(sizer_choice_horizontal,proportion=0,flag =
wx.TOP,border=5)

            #sizer_check_ver.Add(sizer_check2,proportion=0,flag=wx.TOP,border = 5)
            first_name_label = wx.StaticText(self,-1,'First Name',size=(75,-1))
            last_name_label = wx.StaticText(self,-1,'Last Name',size=(75,-1))
            age = wx.StaticText(self,-1,'Balance',size=(75,-1))
            acctype = wx.StaticText(self,-1,'Acctype',size=(75,-1))
            salary = wx.StaticText(self,-1,'Salary',size =(75,-1))
            branch_id = wx.StaticText(self,-1,'branchID',size =(75,-1))
            branch_name = wx.StaticText(self,-1,'branch_name',size =(75,-1))
            branch_zip = wx.StaticText(self,-1,'zip',size =(75,-1))
            emp_id = wx.StaticText(self,-1,'empID',size =(75,-1))
            cus_id = wx.StaticText(self,-1,'customerID',size =(75,-1))
            emp_branch_id = wx.StaticText(self,-1,'empbranchID',size =(75,-1))
            cus_branch_id = wx.StaticText(self,-1,'cusbranchID',size =(75,-1))
            cus_fname = wx.StaticText(self,-1,'custom_fnam',size =(75,-1))
            cus_lname = wx.StaticText(self,-1,'custom_lnam',size =(75,-1))
            self.text_first_name = wx.TextCtrl(self,wx.ID_ANY,"",size=(130,-1))
            self.text_first_name_cond = wx.ComboBox(self,choices =
condition_data,size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_first_name_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_first_name)
            self.text_last_name = wx.TextCtrl(self,-1,"",size=(130,-1))
            self.text_last_name_cond = wx.ComboBox(self,choices =
condition_data,size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_last_name_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_last_name)
            self.text_balance = wx.TextCtrl(self,-1,"",size=(130,-1))
            self.text_balance_cond = wx.ComboBox(self,choices =
['=','<>','<','>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_balance_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_balance)
            self.text_acctype = wx.TextCtrl(self,-1,"",size=(130,-1))
            self.text_acctype_cond = wx.ComboBox(self,choices =
condition_data,size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_acctype_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_acctype)
            self.text_salary = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_salary_cond = wx.ComboBox(self,choices =
['=','<>','>','<',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_salary_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_salary)
            self.text_branch_id = wx.TextCtrl(self,-1,"",size = (130,-1))
```

```
            self.text_branch_id_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_branch_id_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_branch_id)
            self.text_branch_name = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_branch_name_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)

self.text_branch_name_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_branch_name)
            self.text_branch_zip = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_branch_zip_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_branch_zip_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_branch_zip)
            self.text_emp_id = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_emp_id_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_emp_id_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_emp_id)
            self.text_cus_id = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_cus_id_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.text_cus_id_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_cus_id)
            self.text_cus_branch_id = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_cus_branch_id_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)

self.text_cus_branch_id_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_cus_branch_id)
            self.text_emp_branch_id = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_emp_branch_id_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)

self.text_emp_branch_id_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_emp_branch_id)
            self.text_cus_first_name = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_cus_first_name_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)

self.text_cus_first_name_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_emp_branch_id)
            self.text_cus_last_name = wx.TextCtrl(self,-1,"",size = (130,-1))
            self.text_cus_last_name_cond = wx.ComboBox(self,choices =
['=','<>',''],size=(50,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)

self.text_cus_last_name_cond.Bind(wx.EVT_COMBOBOX,self.OnSelect_emp_branch_id)
            sizer_label_text1 = wx.BoxSizer(wx.HORIZONTAL)

sizer_label_text1.Add(first_name_label,proportion=0,flag=wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text1.Add(self.text_first_name_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,
border=5)

sizer_label_text1.Add(self.text_first_name,proportion=0,flag=wx.EXPAND|wx.RIGHT,borde
r=5)
            sizer_label_text2 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text2.Add(last_name_label,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text2.Add(self.text_last_name_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,b
order=5)
```

```
            sizer_label_text2.Add(self.text_last_name,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text3 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text3.Add(age,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text3.Add(self.text_balance_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,bor
der=5)
            sizer_label_text3.Add(self.text_balance,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text4 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text4.Add(acctype,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text4.Add(self.text_acctype_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,bor
der=5)
            sizer_label_text4.Add(self.text_acctype,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text5 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text5.Add(salary,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text5.Add(self.text_salary_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,bord
er=5)
            sizer_label_text5.Add(self.text_salary,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text6 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text6.Add(branch_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text6.Add(self.text_branch_id_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,b
order=5)
            sizer_label_text6.Add(self.text_branch_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text7 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text7.Add(branch_name,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text7.Add(self.text_branch_name_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT
,border=5)
            sizer_label_text7.Add(self.text_branch_name,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text8 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text8.Add(branch_zip,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text8.Add(self.text_branch_zip_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,
border=5)
            sizer_label_text8.Add(self.text_branch_zip,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text9 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text9.Add(emp_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
```

```
sizer_label_text9.Add(self.text_emp_id_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,bord
er=5)
            sizer_label_text9.Add(self.text_emp_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text10 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text10.Add(cus_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text10.Add(self.text_cus_id_cond,proportion=0,flag=wx.EXPAND|wx.RIGHT,bor
der=5)
            sizer_label_text10.Add(self.text_cus_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text11 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text11.Add(cus_branch_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text11.Add(self.text_cus_branch_id_cond,proportion=0,flag=wx.EXPAND|wx.RI
GHT,border=5)
            sizer_label_text11.Add(self.text_cus_branch_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text12 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text12.Add(emp_branch_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text12.Add(self.text_emp_branch_id_cond,proportion=0,flag=wx.EXPAND|wx.RI
GHT,border=5)
            sizer_label_text12.Add(self.text_emp_branch_id,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text13 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text13.Add(cus_fname,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text13.Add(self.text_cus_first_name_cond,proportion=0,flag=wx.EXPAND|wx.R
IGHT,border=5)
            sizer_label_text13.Add(self.text_cus_first_name,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_label_text14 = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text14.Add(cus_lname,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)

sizer_label_text14.Add(self.text_cus_last_name_cond,proportion=0,flag=wx.EXPAND|wx.RI
GHT,border=5)
            sizer_label_text14.Add(self.text_cus_last_name,proportion=0,flag =
wx.EXPAND|wx.RIGHT,border=5)
            sizer_check_ver.Add(sizer_label_text1,proportion=0,flag= wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text2,proportion=0,flag= wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text3,proportion=0,flag= wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text4,proportion=0,flag= wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text5,proportion=0,flag= wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text6,proportion=0,flag= wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text7,proportion=0,flag= wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text8,proportion=0,flag= wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text9,proportion=0,flag= wx.TOP,border=5)
```

```
            sizer_check_ver.Add(sizer_label_text10,proportion=0,flag=
wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text11,proportion=0,flag=
wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text12,proportion=0,flag=
wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text13,proportion=0,flag=
wx.TOP,border=5)
            sizer_check_ver.Add(sizer_label_text14,proportion=0,flag=
wx.TOP,border=5)
            sizer_check_ver.Add(gs_sub,proportion=0,flag=wx.TOP,border=5)
            stat_box = wx.StaticBox(self,label='For Select Options',size=(300,200))
            # BUTTON FOR FINALZIE THE SELECT PART
            self.button_select = wx.Button(self,wx.ID_ANY,'Finalize
Query',size=(100,-1))
            self.button_select.Bind(wx.EVT_BUTTON,self.select_execute)
            #self.button_select.Enable(False)
            self.check_first_name = wx.CheckBox(self,label='First Name')
            self.check_last_name = wx.CheckBox(self,label='Last Name')
            self.check_cus_first_name = wx.CheckBox(self,label='customer_fname')
            self.check_cus_last_name = wx.CheckBox(self,label='customer_lname')
            self.check_balance = wx.CheckBox(self,label='balance')
            self.check_salary= wx.CheckBox(self,label='Salary')
            self.check_acctype = wx.CheckBox(self,label='acctype')
            self.check_emp_branchId = wx.CheckBox(self,label='emp_branchID')
            self.check_cus_branchId = wx.CheckBox(self,label='customer_branchID')
            self.check_emp_Id = wx.CheckBox(self,label='employeeID')
            self.check_cus_Id = wx.CheckBox(self,label='customerID')
            self.check_branch_id_ = wx.CheckBox(self,wx.ID_ANY,label =
'branch_branchID')
            self.check_branch_name_ = wx.CheckBox(self,wx.ID_ANY,label = 'branch
name')
            self.check_branch_zip =  wx.CheckBox(self,wx.ID_ANY,label = 'branch zip')
            sizer_select_super = wx.BoxSizer(wx.VERTICAL)
            sizer_select = wx.StaticBoxSizer(stat_box, wx.VERTICAL)

sizer_select_super.Add(sizer_select,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx.LE
FT,border = 5)

sizer_select.Add(self.check_first_name,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx
.LEFT,border =5)

sizer_select.Add(self.check_last_name,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx.
LEFT,border =5)

sizer_select.Add(self.check_cus_first_name,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGH
T|wx.LEFT,border =5)

sizer_select.Add(self.check_cus_last_name,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT
|wx.LEFT,border =5)

sizer_select.Add(self.check_balance,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx.LE
FT,border =5)
```

```
sizer_select.Add(self.check_salary,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx.LEF
T,border =5)

sizer_select.Add(self.check_acctype,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx.LE
FT,border =5)

sizer_select.Add(self.check_emp_branchId,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|
wx.LEFT,border =5)

sizer_select.Add(self.check_cus_branchId,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|
wx.LEFT,border =5)

sizer_select.Add(self.check_emp_Id,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx.LEF
T,border =5)

sizer_select.Add(self.check_cus_Id,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx.LEF
T,border =5)

sizer_select.Add(self.check_branch_id_,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx
.LEFT,border =5)

sizer_select.Add(self.check_branch_name_,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|
wx.LEFT,border =5)

sizer_select.Add(self.check_branch_zip,proportion=0,flag=wx.EXPAND|wx.TOP|wx.RIGHT|wx
.LEFT,border =5)

sizer_select.Add(self.button_select,proportion=0,flag=wx.TOP|wx.RIGHT|wx.ALIGN_RIGHT,
border=5)

gs.AddMany([(sizer_for_option_vertical,0,wx.EXPAND),(sizer_check_ver,0,wx.EXPAND),(si
zer_select_super,0,wx.EXPAND)])
            self.list_items = ['first_name        last_name        age        salary
ssn    General    Tech    Finance    Hr']
            #self.list_item = wx.ListBox(self,wx.ID_ANY,size=(200,400),choices =
self.list_items,style = wx.LB_SINGLE)
            stat_box2 = wx.StaticBox(self,wx.ID_ANY,label='Display Selected
Data',size=(600,225))
            sizer_display = wx.StaticBoxSizer(stat_box2, wx.VERTICAL)
            self.gridI = wx.grid.Grid(self)
            self.tableI = gridTable(100,14)
            self.gridI.SetTable(self.tableI,True)
            #self.table.SetValue(1,1,'sabbir')
            #self.table.SetValue(2,2,'Cold Play')
            self.gridI.SetColSize(0,150)
            self.gridI.SetColSize(1,150)
            self.gridI.SetColSize(2,150)
            self.gridI.SetColSize(3,150)
            self.gridI.SetColSize(4,150)
            #self.grid.DeleteRows( 2,1)
            #self.grid.ForceRefresh()
            sizer_display.Add(self.gridI,proportion=0,flag=
wx.EXPAND|wx.TOP|wx.BOTTOM|wx.LEFT|wx.RIGHT,border=0)
            sizer_for_panels = wx.BoxSizer(wx.VERTICAL)
```

```
            sizer_for_panels.Add(gs,proportion = 0, flag =
wx.EXPAND|wx.TOP|wx.BOTTOM|wx.LEFT|wx.RIGHT,border = 5)

sizer_for_panels.Add(sizer_display,proportion=0,flag=wx.EXPAND|wx.TOP|wx.BOTTOM|wx.LE
FT|wx.RIGHT,border=5)
            self.SetSizer(sizer_for_panels)
            return
        def select_and(self,e):
            return
        def select_or(self,e):
            return

        def OnSelect_branch_id(self,e):
            self.branch_id_cond = e.GetString()
            return
        def OnSelect_branch_name(self,e):
            self.branch_name_cond = e.GetString()
            return
        def OnSelect_branch_zip(self,e):
            self.branch_zip_cond = e.GetString()
            return
        def OnSelect_cus_id(self,e):
            self.cus_id_cond = e.GetString()
            return
        def OnSelect_cus_branch_id(self,e):
            self.cus_branch_id_cond = e.GetString()
            return
        def OnSelect_emp_id(self,e):
            self.emp_id_cond = e.GetString()
            return
        def OnSelect_emp_branch_id(self,e):
            self.emp_branch_id_cond = e.GetString()
            return
        def OnSelect_acctype(self,e):
            self.acctyp_cond = e.GetString()
            return
        def OnSelect_salary(self,e):
            self.salary_cond = e.GetString()
            return
        def OnSelect_balance(self,e):
            self.balance_cond = e.GetString()
            return
        def OnSelect_first_name(self,e):
            self.first_name_cond = e.GetString()
            return
        def OnSelect_last_name(self,e):
            self.last_name_cond = e.GetString()
            return
        def matchAccess(self,l1,l2):
            notzero = 0
            for check in l1:
                if(check == 1):
                    notzero += 1
            if(notzero == 0):
```

```
                self.showDialog("You can not access any of the information you are
blogged")
                return False
            if(l2[0] == 1):
                return True
            for elem in range(0,len(l1)-1):
                if(l1[elem] == 1):
                    if(l1[elem] == l2[elem]):
                        return True
            return False
        def select_event_handler(self,event):
            self.data_list = self.parent.objectToWork.data[0][2:]
            print(self.data_list)
            con = getConnector(username = 'root', passwd='ztwitasd4',
machine='localhost',databaseName= 'twitter_sabbir',port= 3306)
            operator = con.cursor()
            stringTo = 'select * from information'
            operator.execute(stringTo)
            self.dict_feature = {2:0,3:0,4:0,5:0,6:0}
            if(self.check_first_name_del.GetValue() == True):
                self.dict_feature[2] = 1
            if(self.check_last_name_del.GetValue() == True):
                self.dict_feature[3] = 1
            if(self.check_age_del.GetValue() == True):
                self.dict_feature[4] = 1
            if(self.check_salary_del.GetValue() == True):
                self.dict_feature[5] = 1
            if(self.check_ssn_del.GetValue() == True):
                self.dict_feature[6] = 1
            data = operator.fetchall()
            print(data)
            self.parentList = []
            if(len(data) ==0):
                self.showDialog("There is no data to display")
                return None
            for row in data:
                tempList= []
                for col in row:
                    tempList.append(col)
                self.parentList.append(tempList)
            self.addToGrid(self.grid_ob)
            operator.close()
            con.close()
            return
        def get_feature_extraction_from_tokens(self,tokens,diversity):
            diverse_dict = {}
            for each in tokens:
                if(diversity.get(each,None)!= None):
                    diverse_dict[each] = 1
                else:
                    diverse_dict[each] = 0
            return
        def addToGrid(self,grids):
            row_number = 0
            for elem in self.parentList:
```

```
                tempList = elem[7:]
                print(tempList)
                if(self.matchAccess(self.data_list,tempList) == False):
                    continue
                for col in range(0,len(elem)):
                    if(self.dict_feature.get(col,None) != None):
                        if(self.dict_feature[col] == 0):
                                elem[col] = 'None'
                grids.addItem(row_number,elem)
                row_number += 1
            return
        def tab_page_two(self):
            fgs = wx.FlexGridSizer(1,3,5,15)
            stat_box1 = wx.StaticBox(self,wx.ID_ANY,'Select Options',size=(350,200))
            stat_box2 = wx.StaticBox(self,wx.ID_ANY,'Delete Data
Source',size=(250,200))
            # sizer = wx.BoxSizer(wx.VERTICAL)
            # sizerh = wx.BoxSizer(wx.HORIZONTAL)
            label2 = wx.StaticText(self,-1,'Select Data to Populate Grid',size=(180,-
1),style = wx.ALIGN_RIGHT)
            self.button_select_populate =
wx.Button(self,wx.ID_ANY,'Execute',size=(100,-1))
            self.button_select_populate.Bind(wx.EVT_BUTTON,self.select_event_handler)
            self.button_select_and_del =
wx.Button(self,wx.ID_ANY,'Delete',size=(100,-1))
            self.check_first_name_del = wx.CheckBox(self,wx.ID_ANY,label = 'First
Name')
            self.check_last_name_del = wx.CheckBox(self,wx.ID_ANY,label = 'Last
Name')
            self.check_age_del = wx.CheckBox(self,wx.ID_ANY,label = 'Salary')
            self.check_salary_del = wx.CheckBox(self,wx.ID_ANY,label = 'Balance')
            self.check_ssn_del = wx.CheckBox(self,wx.ID_ANY,label = 'Acctype')

            sizer_select = wx.StaticBoxSizer(stat_box1,wx.VERTICAL)
            sizer_select_del = wx.StaticBoxSizer(stat_box2,wx.VERTICAL)

sizer_select.Add(label2,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT,border=5)

sizer_select.Add(self.check_first_name_del,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGH
T,border=5)

sizer_select.Add(self.check_last_name_del,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT
,border=5)

sizer_select.Add(self.check_age_del,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT,borde
r=5)

sizer_select.Add(self.check_salary_del,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT,bo
rder=5)

sizer_select.Add(self.check_ssn_del,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT,borde
r=5)
```

```
sizer_select.Add(self.button_select_populate,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT|wx.ALIGN_RIGHT,border=5)
            label1 = wx.StaticText(self,-1,'Select a Field To Delete From Grid',size=(180,-1),style = wx.ALIGN_RIGHT)
            self.grid_ob = createGrid(self,50,7)
            self.grid_del = self.grid_ob.getGrid()

            ver_box = wx.BoxSizer(wx.HORIZONTAL)
            sizer_select_del.Add(label1,proportion = 0, flag = wx.BOTTOM|wx.TOP|wx.LEFT|wx.RIGHT,border=5)

sizer_select_del.Add(self.grid_del,proportion=1,flag=wx.BOTTOM|wx.TOP|wx.LEFT|wx.RIGHT,border=5)

sizer_select_del.Add(self.button_select_and_del,proportion=0,flag=wx.BOTTOM|wx.TOP|wx.LEFT|wx.RIGHT|wx.ALIGN_RIGHT,border=5)
            self.button_select_and_del.Bind(wx.EVT_BUTTON,self.del_event_handler)
            fgs.AddMany([(sizer_select,0,wx.EXPAND),(sizer_select_del,0,wx.EXPAND)])
            fgs.AddGrowableCol(1,1)
            fgs.AddGrowableCol(0,1)
            fgs.AddGrowableRow(0,1)
            ver_box.Add(fgs,proportion=1,flag = wx.EXPAND|wx.BOTTOM|wx.LEFT|wx.RIGHT|wx.TOP,border = 15)
            #txtOne = wx.TextCtrl(self, wx.ID_ANY, "",size = (180,-1))
__________# button1 = wx.Button(self,wx.ID_ANY,'Add')
__________# button2 = wx.Button(self,wx.ID_ANY,'Add')
            #sizerh.Add(label1,proportion=1,flag = wx.RIGHT,border=5)
            #sizerh.Add(txtOne,proportion=1,flag = wx.LEFT|wx.ALIGN_CENTER, border = 5)
            #sizerh.Add(button1,proportion=1,flag = wx.RIGHT|wx.LEFT|wx.ALIGN_RIGHT, border = 5)
__________# txtTwo = wx.TextCtrl(self, wx.ID_ANY, "",size=(180,-1))
__________# sizer = wx.BoxSizer(wx.VERTICAL)
__________# sizer.Add(sizerh, 0, wx.ALL, 5)
__________# sizer.Add(txtTwo, 0, wx.ALL, 5)
            self.SetSizer(ver_box)
            return
        def del_event_handler(self,event):
            row_number = self.grid_del.GetSelectedRows()
            print(row_number)
            item_to_del = self.parentList[row_number[0]]
            conn = getConnector(machine='localhost', passwd='ztwitasd4', username='root', databaseName='twitter_sabbir', port=3306)
            operator = conn.cursor()
            print(str(item_to_del[0]))
            string_To_execute = 'delete from information where id = '+ str(item_to_del[0])
            try:
                operator.execute(string_To_execute)
                conn.commit()
                success = 1
                print('execution done')
            except Exception:
                print(Exception.message)
```

```
                success = 0
                conn.rollback()
            if(success == 1):
                self.showDialog("Data Successfully Deleted")
            else:
                self.showDialog("May be you are not permitted to Delete this
information")
                return
            del self.parentList[row_number[0]]
            self.grid_del.DeleteRows(row_number[0],1)
            self.grid_del.AppendRows(numRows = 1)

            #self.addToGrid()
            operator.close()
            conn.close()
            return

        def OnSelect_file(self,e):
            if(self.get_scope =='global'):
                Mbox('Warning', 'You can not select files while in global mode',
style=wx.FONTFAMILY_DEFAULT)
                return
            if(len(self.cb_tab.GetValue()) == 0 ):
                self.showDialog("Select Appropriate Table")
                return
            dlg = wx.FileDialog(self, message="Open an Image...",
defaultDir=os.getcwd(),defaultFile="", style=wx.OPEN)
            if(dlg.ShowModal() == wx.ID_OK):
                self.text_file_browser.SetValue(dlg.GetPath())
            list_components = self.text_file_browser.GetValue().split('\\')
            self.file_to_read =''
            for each in xrange(len(list_components) - 1):
                self.file_to_read =self.file_to_read + list_components[each] +'\\'
            self.file_to_read += list_components[len(list_components) - 1]
            print(self.file_to_read)
            return
        def init_gridI_1(self):

            for i in range(0,self.grid.GetNumberRows()):
                for j in range(0,self.grid.GetNumberCols()):
                    self.table.SetValue(i,j,"")
            return
        def populate_grid1(self,dataOb):
            self.init_gridI_1()
            row_number = 0
            #if(self.cb1.GetValue() == 'Select'):
            for data in dataOb.data:
                #zipper = zip(list_list,data)
                #tempStr = ""
                #temp_list_op = data[7:]
                #if(self.matchAccess(auth_list, temp_list_op) ==False):
                #   continue
                col_number = 0
                print('i am here')
                for (key,value) in self.dict_list_1_1.items():
```

```
                    if(value != 0):
                        print('key = '  + key + 'value = ' + str(value))
                        print('item index = ' + str(self.item_index_1[key]))
                        self.table.SetValue(row_number,self.dict_list_1[key]-
1,data[self.item_index_1[key]])
                    else:
                        self.table.SetValue(row_number,self.dict_list_1[key]-
1,'None')
                        col_number += 1
                row_number += 1
                #self.list_item.Append(tempStr)
                # self.list_item.Append(temp_list)
            self.grid.ForceRefresh()
            return
        def set_ordering(self,dict_tab11):
            indexing = 0
            for keys in dict_tab11:
                    self.dict_list_1_1[keys] = 1
                    self.item_index_1[keys] = dict_tab11[keys]-1
                    indexing += 1
            return
        def determine_order(self,query_exe):
            self.dict_list_1_1 =
{'emplname':0,'empfname':0,'balance':0,'salary':0,'acctype':0,'customerID':0,'employe
eID':0,'employee.branchID':0,'customer.branchID':0,'branch.branchID':0,'name_state':0
,'zip':0,'first_name':0,'last_name':0}
            self.dict_list_1 =
{'emplname':2,'empfname':1,'balance':3,'salary':4,'acctype':5,'employee.branchID':6,'
customer.branchID':7,'branch.branchID':8,'name_state':9,'zip':10,'customerID':11,'emp
loyeeID':12,'first_name':13,'last_name':14}
            self.item_index_1 =
{'emplname':0,'empfname':0,'balance':0,'salary':0,'acctype':0,'employee.branchID':0,'
customer.branchID':0,'branch.branchID':0,'name_state':0,'zip':0,'customerID':0,'emplo
yeeID':0,'first_name':0,'last_name':0}
            work_with = query_exe.split('from')
            if( work_with[0].find('select')!= -1):
                work_with[0] = work_with[0].replace('select','')
            elif(work_with[0].find('Select')!= -1):
                work_with[0] = work_with[0].replace('Select','')
            working = work_with[0].strip().split(',')
            indexing = 0
            print(query_exe)
            if(working[0] =='*'):
                if(query_exe.find( 'branch') != -1):
                    self.set_ordering(self.dict_tab11)
                elif(query_exe.find('customer') != -1):
                    self.set_ordering(self.dict_tab22)
                else:
                    self.set_ordering(self.dict_tab33)
                return

            for each in working:
                if(len(each.strip()) != 0):
                    self.dict_list_1_1[each.strip()] = 1
                    self.item_index_1[each.strip()] = indexing
```

```
                    indexing += 1
            return
        def decide_on_operation(self,checker):
            if(isinstance(checker,str) ==True):
                self.showDialog(checker)
            else:
                self.showDialog('Data inserted successfully')
            return
        def OnExecute_query(self,e):
            if(self.get_scope =='global'):
                Mbox('Warning', 'You can not execute local query while in global
mode', style=wx.FONTFAMILY_DEFAULT)
                return
            conn = getConnector(machine = 'localhost',username = 'root',passwd =
'ztwitasd4',port =3306,databaseName='twitter_sabbir')
            self.dict_tab11 = {'branch.branchID':1,'name_state':2,'zip':3}
            self.dict_tab22 =
{'customerID':1,'customer.branchID':2,'first_name':3,'last_name':4,'acctype':5,'balan
ce':6}
            self.dict_tab33 =
{'employeeID':1,'employee.branchID':2,'emplname':3,'empfname':4,'salary':5}
            if(self.check_technology_per.GetValue() == False and
self.check_humanresource_per.GetValue() == False ):
                self.showDialog('you have to select an option either upload or
execute query')
                return
            if(self.check_humanresource_per.GetValue() == True):
                if(len(self.text_query.GetValue().strip()) == 0):
                    self.showDialog('get appropriate file')
                    return
            if(self.check_technology_per.GetValue() == True):
                if(len(self.file_to_read) == 0):
                    self.showDialog('enter the proper file name')
                    return
                self.file_ob = open(self.file_to_read)
                try:
                    self.data = self.file_ob.readlines()
                finally:
                    self.file_ob.close()
                if(self.cb_tab.GetValue() == 'branch'):
                    sub_data = self.data[0].split('\t')
                    print(sub_data)
                    for each in sub_data:
                        if(self.dict_tab11.get(each.strip(),None) == None):
                            self.showDialog('your data format is wrong give a correct
format as input for branch')
                            return

                    checker = insertData('branch', conn, self.data[1:])
                elif(self.cb_tab.GetValue() == 'customer'):
                    sub_data = self.data[0].split('\t')
                    for each in sub_data:
                        if(self.dict_tab22.get(each.strip(),None) == None):
                            self.showDialog('your data format is wrong give a correct
format as input for customer')
```

```
                            return

                        checker = insertData('customer', conn, self.data[1:])
                    else:
                        sub_data = self.data[0].split('\t')
                        for each in sub_data:
                            if(self.dict_tab33.get(each.strip(),None) == None):
                                self.showDialog('your data format is wrong give a correct
format as input for employee')
                                return

                        checker = insertData('employee', conn, self.data[1:])
                    self.decide_on_operation(checker)
                else:
                    self.query_execution = self.text_query.GetValue().strip()
                    #print(self.query_execution)
                    #try:
                    print(str(len(self.parent.objectToWork.data)))
                    if(self.query_execution.lower().find('select') != -1):
                            self.determine_order(self.query_execution)
                            print(self.query_execution)
                            temp_query = self.query_execution.decode('utf-8','ignore')
                            if(temp_query.lower().find('customer') != -1):
                                temp_query =
re.sub('customer','customer_'+str(self.parent.objectToWork.data[0][3]),temp_query)
                                temp_query = re.sub('customer_[0-
9]+ID','customerID',temp_query)
                            if(temp_query.lower().find('employee') != -1):
                                temp_query =
re.sub('employee','employee_'+str(self.parent.objectToWork.data[0][3]),temp_query)
                                temp_query = re.sub('employee_[0-
9]+ID','employeeID',temp_query)
                            print(temp_query)
                            self.data_list_query = fetchData(conn,temp_query)
                            if(isinstance(self.data_list_query,str) == False):
                                self.populate_grid1(self.data_list_query)
                            else:
                                self.decide_on_operation(self.data_list_query)

                    else:
                                suc =
dbHW1.other_query_execution(self.query_execution.decode('utf-8','ignore'),conn)
                                if(isinstance(suc,str) == False):
                                    self.showDialog('successfully executed')
                                else:
                                    self.decide_on_operation(suc)
                    #except Exception:
                        # print(Exception.message)
                        #self.showDialog('query is not right')
                        #return
                return
            def tab_page_three(self):
                gs = wx.FlexGridSizer(1,2,5,15)#(1,3,5,5)
                stat_box1 = wx.StaticBox(self,wx.ID_ANY,'Data Upload and Query
Exe',size=(300,200))
```

```
            #stat_box2 = wx.StaticBox(self,wx.ID_ANY,'Where Options',size =
(300,200))
            '''self.check_first_name_per = wx.CheckBox(self,wx.ID_ANY,label = 'First
Name')
            self.check_last_name_per = wx.CheckBox(self,wx.ID_ANY,label = 'Last
Name')
            self.check_age_per = wx.CheckBox(self,wx.ID_ANY,label = 'Age')
            self.check_salary_per = wx.CheckBox(self,wx.ID_ANY,label = 'Salary')
            self.check_ssn_per = wx.CheckBox(self,wx.ID_ANY,label = 'SSN')'''
            sizer_select = wx.StaticBoxSizer(stat_box1,wx.VERTICAL)

'''sizer_select.Add(self.check_first_name_per,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.R
IGHT,border=5)

sizer_select.Add(self.check_last_name_per,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT
,border=5)

sizer_select.Add(self.check_age_per,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT,borde
r=5)

sizer_select.Add(self.check_salary_per,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT,bo
rder=5)

sizer_select.Add(self.check_ssn_per,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT,borde
r=5)'''
            #label_text_1 = wx.StaticText(self,-1,'Select Data to Populate
Grid',size=(180,-1),style = wx.ALIGN_RIGHT)
            browse_to_file_label = wx.StaticText(self,-1,'Browse and Select
File',size=(150,-1))

sizer_select.Add(browse_to_file_label,proportion=0,flag=wx.TOP|wx.BOTTOM|wx.RIGHT|wx.
LEFT,border= 5)
            self.text_file_browser = wx.TextCtrl(self,wx.ID_ANY,"",size=(450,-1))
            self.browse_file = wx.Button(self,wx.ID_ANY,'Browse',size=(75,22))
            self.browse_file.Bind(wx.EVT_BUTTON,self.OnSelect_file)
            hor_browse = wx.BoxSizer(wx.HORIZONTAL)
            hor_browse.Add(self.text_file_browser,proportion=0,flag =
wx.RIGHT|wx.BOTTOM|wx.TOP|wx.LEFT,border=5)
            hor_browse.Add(self.browse_file,proportion=0,flag = wx.ALIGN_BOTTOM|
wx.RIGHT|wx.BOTTOM|wx.TOP|wx.LEFT,border=5)
            sizer_select.Add(hor_browse,proportion=0,flag = wx.BOTTOM,border = 5)
            self.text_query = wx.TextCtrl(self,-1,"",size=(450,-1))
            #self.text_age_per = wx.TextCtrl(self,-1,"",size=(200,-1))
            #self.text_ssn_per = wx.TextCtrl(self,-1,"",size=(200,-1))
            #self.text_salary_per = wx.TextCtrl(self,-1,"",size = (200,-1))
            write_query_label = wx.StaticText(self,-1,'Query Plan',size=(75,-1))
            sizer_select.Add(write_query_label,proportion = 0,flag =
wx.BOTTOM|wx.LEFT|wx.RIGHT,border= 5)
            sizer_select.Add(self.text_query,proportion = 0,flag =
wx.BOTTOM|wx.LEFT|wx.RIGHT|wx.TOP,border= 5 )

            #age = wx.StaticText(self,-1,'Age',size=(75,-1))
            #ssn = wx.StaticText(self,-1,'SSN',size=(75,-1))
            #salary = wx.StaticText(self,-1,'Salary',size =(75,-1))
            self.button_select_per = wx.Button(self,wx.ID_ANY,'Execute',size=(75,-1))
```

```
            self.button_select_per.Bind(wx.EVT_BUTTON,self.OnExecute_query)
            '''sizer_label_text1_per = wx.BoxSizer(wx.HORIZONTAL)

sizer_label_text1_per.Add(first_name_label,proportion=0,flag=wx.RIGHT|wx.TOP|wx.BOTTO
M|wx.LEFT,border=5)

sizer_label_text1_per.Add(self.text_first_name_per,proportion=0,flag=wx.RIGHT|wx.TOP|
wx.BOTTOM|wx.LEFT,border=5)
            sizer_label_text2_per = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text2_per.Add(last_name_label,proportion=0,flag =
wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT,border=5)
            sizer_label_text2_per.Add(self.text_last_name_per,proportion=0,flag =
wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT,border=5)
            sizer_label_text3_per = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text3_per.Add(age,proportion=0,flag =
wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT,border=5)
            sizer_label_text3_per.Add(self.text_age_per,proportion=0,flag =
wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT,border=5)
            sizer_label_text4_per = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text4_per.Add(ssn,proportion=0,flag =
wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT,border=5)
            sizer_label_text4_per.Add(self.text_ssn_per,proportion=0,flag =
wx.EXPAND|wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT,border=5)
            sizer_label_text5_per = wx.BoxSizer(wx.HORIZONTAL)
            sizer_label_text5_per.Add(salary,proportion=0,flag =
wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT,border=5)
            sizer_label_text5_per.Add(self.text_salary_per,proportion=0,flag =
wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT,border=5)
            sizer_ver = wx.StaticBoxSizer(stat_box2,wx.VERTICAL)
            sizer_ver.Add(sizer_label_text1_per)
            sizer_ver.Add(sizer_label_text2_per)
            sizer_ver.Add(sizer_label_text3_per)
            sizer_ver.Add(sizer_label_text4_per)
            sizer_ver.Add(sizer_label_text5_per)'''
            box_sizer = wx.BoxSizer(wx.VERTICAL)
            stat_box3 = wx.StaticBox(self,wx.ID_ANY,'Select Options',size =
(250,200))
            sizer_ver_for_compartments = wx.StaticBoxSizer(stat_box3,wx.VERTICAL)
            self.check_technology_per = wx.RadioButton(self,wx.ID_ANY,label = 'Upload
Data')
            self.check_humanresource_per = wx.RadioButton(self,wx.ID_ANY,label =
'Execute Query')
            self.cb_tab = wx.ComboBox(self,choices =
['branch','employee','customer'],size=(150,-1),style = wx.CB_READONLY|wx.ALIGN_RIGHT)
            self.cb_tab.Bind(wx.EVT_COMBOBOX,self.OnSelect_tab)
            labelCb = wx.StaticText(self,-1,'Select Table',size=(50,-1))
            #self.check_general_per = wx.CheckBox(self,wx.ID_ANY,label = 'General')
            #self.check_finance_per = wx.CheckBox(self,wx.ID_ANY,label = 'Finance')
            #sizer_ver_for_compartments = wx.StaticBoxSizer(stat_box1,wx.VERTICAL)
            hor_tab = wx.BoxSizer(wx.HORIZONTAL)
            hor_tab.Add(labelCb,proportion=0,flag = wx.RIGHT, border=5)
            hor_tab.Add(self.cb_tab,proportion=0,flag=wx.RIGHT,border = 5)

sizer_ver_for_compartments.Add(hor_tab,proportion=0,flag=wx.RIGHT|wx.TOP|wx.BOTTOM|wx
.LEFT,border = 5)
```

```
sizer_ver_for_compartments.Add(self.check_technology_per,proportion=0,flag =
wx.RIGHT|wx.TOP|wx.BOTTOM|wx.LEFT, border=5)

sizer_ver_for_compartments.Add(self.check_humanresource_per,proportion=0,flag=wx.RIGH
T|wx.TOP|wx.BOTTOM|wx.LEFT,border = 5)

#sizer_ver_for_compartments.Add(self.check_general_per,proportion=0,flag=wx.RIGHT|wx.
TOP|wx.BOTTOM|wx.LEFT,border=5)

#sizer_ver_for_compartments.Add(self.check_finance_per,proportion=0,flag=wx.RIGHT|wx.
TOP|wx.BOTTOM|wx.LEFT,border=5)
            stat_box_execute = wx.BoxSizer(wx.HORIZONTAL)
            stat_box_execute.Add(wx.StaticText(self,wx.ID_ANY,'         '))
            stat_box_execute.Add(self.button_select_per,wx.ID_ANY,flag
=wx.EXPAND|wx.TOP|wx.ALIGN_RIGHT|wx.BOTTOM|wx.RIGHT,border=5)

gs.AddMany([(sizer_select,0,wx.EXPAND),(sizer_ver_for_compartments,0,wx.EXPAND)])#'''
(sizer_ver,0,wx.EXPAND),'''
            gs1 = wx.GridSizer(1,3,5,20)
            box_sizer.Add(gs,proportion=0,flag =
wx.EXPAND|wx.TOP|wx.BOTTOM|wx.LEFT|wx.RIGHT,border=5)
            stat_box_list = wx.StaticBox(self,wx.ID_ANY, label='List of Data',size =
(400,200))
            stat_box_list_sizer = wx.StaticBoxSizer(stat_box_list,wx.VERTICAL)

#box_sizer.Add(stat_box_execute,proportion=0,flag=wx.EXPAND|wx.TOP|wx.BOTTOM|wx.RIGHT
,border=5)

gs1.AddMany([(wx.StaticText(self),wx.EXPAND),(wx.StaticText(self),wx.EXPAND),(stat_bo
x_execute,0,wx.EXPAND)])
            #gs1.AddGrowableCol(0,1)
            gs.AddGrowableCol(0,2)

box_sizer.Add(gs1,proportion=0,flag=wx.EXPAND|wx.LEFT|wx.TOP|wx.BOTTOM|wx.RIGHT,borde
r=5)
            self.grid = wx.grid.Grid(self)
            self.table = gridTable(100,14)
            self.grid.SetTable(self.table,True)
            #self.table.SetValue(1,1,'sabbir')
            #self.table.SetValue(2,2,'Cold Play')
            self.grid.SetColSize(0,150)
            self.grid.SetColSize(1,150)
            self.grid.SetColSize(2,150)
            self.grid.SetColSize(3,150)
            self.grid.SetColSize(4,150)
            self.grid.DeleteRows( 2,1)
            self.grid.ForceRefresh()

stat_box_list_sizer.Add(self.grid,proportion=1,flag=wx.EXPAND|wx.TOP|wx.BOTTOM|wx.LEF
T|wx.RIGHT,border =1)

box_sizer.Add(stat_box_list_sizer,proportion=1,flag=wx.EXPAND|wx.TOP|wx.BOTTOM|wx.LEF
T|wx.RIGHT,border = 1)
            self.SetSizer(box_sizer)
```

```
            return
        def OnSelect_tab(self,e):
            val = e.GetString()
            return
        def getSetVal(self,setVal):
            permit_list2 = self.list_of_users.GetChecked()
            if(len(permit_list2) == 0):
                self.showDialog("Select At least One data recored")
                return
            #print(permit_list)
            print(permit_list2)
            tech = self.check_technology_per_usr.GetValue()
            hr = self.check_humanresource_per_usr.GetValue()
            gen = self.check_general_per_usr.GetValue()
            finance = self.check_finance_per_usr.GetValue()
            if(tech==False and hr==False and gen==False and finance==False):
                self.showDialog("Select At least One compartments")
                return
            con = getConnector(username='root', passwd='ztwitasd4',
machine='localhost', databaseName='twitter_sabbir', port=3306)
            operator = con.cursor()
            for item in permit_list2:
                string_to = self.list_of_data_per[item]
                get_id = string_to.split('\t')
                tempS1 = re.search('\'[0-9]*\'', get_id[0])
                tempL = tempS1.group().split('\'')
                print(get_id)
                if(tech== True):
                    stringToexec = 'update information set Dept2 = ' + str(setVal)+'
where id = ' +tempL[1]
                    print(stringToexec)
                    result = self.call_for_execution(stringToexec,operator,con)
                    if(result == 0):
                        operator.close()
                        con.close()
                        self.showDialog("Couldn't commit there is a problem press
Ok")
                        return
                if(gen == True):
                    stringToexec = 'update information set Dept1 = ' + str(setVal)+'
where id = ' +tempL[1]
                    result = self.call_for_execution(stringToexec,operator,con)
                    if(result == 0):
                        self.showDialog("Couldn't commit there is a problem press
Ok")
                        operator.close()
                        con.close()
                        return
                if(hr == True):
                    stringToexec = 'update information set Dept4 = ' + str(setVal)+'
where id = ' +tempL[1]
                    result = self.call_for_execution(stringToexec,operator,con)
                    if(result == 0):
```

```
                        self.showDialog("Couldn't commit there is a problem press
Ok")
                        operator.close()
                        con.close()
                        return
                if(finance ==True):
                    stringToexec = 'update information set Dept3 = ' + str(setVal)+'
where id = ' +tempL[1]
                    result = self.call_for_execution(stringToexec,operator,con)
                    if(result == 0):
                        self.showDialog("Couldn't commit there is a problem press
Ok")
                        operator.close()
                        con.close()
                        return

            self.showDialog("Successfully Updated")
            operator.close()
            con.close()
            self.get_list_of_data()
            self.list_of_users.Clear()
            self.check_technology_per_usr.SetValue(False)
            self.check_general_per_usr.SetValue(False)
            self.check_finance_per_usr.SetValue(False)
            self.check_humanresource_per_usr.SetValue(False)
            for elem in self.list_of_data_per:
                self.list_of_users.Append(elem)
            return
        def permit_grant(self,event):
            #permit_list = self.list_of_users.GetSelections()
            self.getSetVal(True)
            return
        def showDialog(self,statement):
            dlg = wx.MessageDialog(self,
               statement,
               "Confirm", wx.OK|wx.ICON_QUESTION)
            result = dlg.ShowModal()
            dlg.Destroy()
            if result == wx.ID_OK:
                pass
                # conn.close()

            return
        def call_for_execution(self,stringToexec,operator,con):
            success = 0
            try:
                operator.execute(stringToexec)
                con.commit()
                success = 1
            except Exception:
                print(Exception.message)
                con.rollback()
                success = 0
            return success
        def permit_revoke(self,event):
```

```
            self.getSetVal(False)
            return
        def get_list_of_data(self):
            con = getConnector(machine = 'localhost', username = 'root', passwd =
'ztwitasd4', databaseName='twitter_sabbir', port=3306)
            operator = con.cursor()
            stringToexecute = 'select * from information'
            operator.execute(stringToexecute)
            data = operator.fetchall()
            print(data)
            self.list_of_data_per = []
            dict_feature_name = {0:'id',1:'username',2:'first name',3:'last
name',4:'age',5:'salary',6:'ssn',7:'gen',8:'tech',9:'finance',10:'hr'}
            if(len(data) ==0):
                return None
            for row in data:
                tempList= ''
                i1 = 0
                for col in row:
                    tempList += str((dict_feature_name[i1],str(col)))+'\t'
                    i1 += 1
                self.list_of_data_per.append(tempList)
            operator.close()
            con.close()
            return
```